

Curso Completo de Programación en Python

Descripción: Este curso proporciona una introducción exhaustiva al lenguaje de programación Python, cubriendo desde los fundamentos básicos hasta conceptos avanzados como la programación orientada a objetos y el manejo de archivos, preparando a los participantes para desarrollar aplicaciones robustas y eficientes.

Módulo 1: Módulo 2: Estructuras de Control y Funciones

1.1 Lección 2.1: Sentencias Condicionales (if/elif/else)

¡Excelente! Es un placer guiarte en este viaje por la programación. Prepárate para dominar una de las herramientas más fundamentales para dar "inteligencia" a tus programas.

Lección 2.1: Sentencias Condicionales (if/elif/else)

Introducción al Control de Flujo

Hasta ahora, hemos visto cómo un programa puede ejecutar una serie de instrucciones de forma secuencial, una tras otra. Sin embargo, la verdadera potencia de la programación reside en la capacidad de tomar decisiones. ¿Qué ocurre si queremos que nuestro programa se comporte de una manera diferente según ciertas circunstancias? ¿O que ejecute un bloque de código solo si se cumple una condición específica?

Aquí es donde entran en juego las **sentencias condicionales**. Son la base para implementar la lógica de decisión en nuestros programas, permitiéndoles adaptarse a diferentes escenarios y entradas. Gracias a ellas, podemos controlar el **flujo de ejecución**, decidiendo qué camino tomará nuestro código.

Para comprender y aplicar las sentencias condicionales, es crucial recordar los **tipos booleanos** (True/False) y los **operadores relacionales** (==, !=, <, >, <=, >=) y **lógicos** (and, or, not) que aprendimos previamente. Estos nos permitirán construir las condiciones que nuestro programa evaluará.

Objetivo de la Lección: Al finalizar esta lección, serás capaz de implementar lógica condicional utilizando if, elif y else para controlar el flujo de ejecución de tus programas, basándote en diferentes escenarios y condiciones.

1. La Sentencia if (Si...)

La sentencia if es la forma más básica de un condicional. Permite ejecutar un bloque de código **solamente si una condición específica es verdadera**.

Sintaxis:

```
python if condicion: # Este bloque de código se ejecuta si 'condicion' es Verdadera (True)
sentencia_1
sentencia_2 # ...
```

Puntos Clave: * La condicion debe ser una expresión que evalúe a un valor booleano (True o False). * Los dos puntos (:) al final de la línea if son obligatorios. * El bloque de código a ejecutar debe estar **indentado** (generalmente con 4 espacios) con respecto a la línea if. La indentación es crucial en Python para definir bloques de código.

Ejemplo Práctico:

Imaginemos que queremos saludar a un usuario si su sitio web favorito es "mundogeek.net".

```
```python
```

### Definimos el sitio web favorito

---

```
fav = "mundogeek.net"
```

## Evaluamos la condición: ¿fav es igual a "mundogeeek.net"?

---

```
if fav == "mundogeeek.net": print("¡Tienes buen gusto!") print("Gracias por visitar nuestro sitio.")

print("Fin del programa.") ``
```

**¿Qué sucede aquí?** 1. La variable fav se inicializa con "mundogeeek.net". 2. La condición fav == "mundogeeek.net" se evalúa. Como "mundogeeek.net" es igual a "mundogeeek.net", la condición es True. 3. Dado que la condición es True, el bloque de código indentado (print("¡Tienes buen gusto!") y print("Gracias por visitar nuestro sitio.)) se ejecuta. 4. Finalmente, se ejecuta print("Fin del programa.") ya que está fuera del bloque if.

### ¿Y si la condición fuera falsa?

```
``python fav = "otro_sitio.com"

if fav == "mundogeeek.net": print("¡Tienes buen gusto!") print("Gracias por visitar nuestro sitio.")

print("Fin del programa.") ``
```

En este caso, fav == "mundogeeek.net" sería False, por lo que el bloque indentado bajo el if **no se ejecutaría**. Solo veríamos Fin del programa. en la consola.

---

## 2. La Sentencia if/else (Si... Sino...)

---

A menudo, no solo queremos ejecutar código si una condición es verdadera, sino que también queremos ejecutar un *código diferente* si la condición es falsa. Para esto, utilizamos la sentencia else.

### Sintaxis:

```
python if condicion: # Este bloque se ejecuta si 'condicion' es Verdadera sentencia_si_verdadera_1
sentencia_si_verdadera_2 else: # Este bloque se ejecuta si 'condicion' es Falsa sentencia_si_falsa_1
sentencia_si_falsa_2
```

**Puntos Clave:** \* El bloque else no lleva ninguna condición, ya que se ejecuta por defecto si la condición del if (y de cualquier elif previo) es False. \* Al igual que con if, else debe ir seguido de dos puntos (:) y su bloque de código debe estar indentado.

### Ejemplo Práctico:

Comprobemos si un número es par o impar.

```
``python numero = 15

if numero % 2 == 0: # El operador % (módulo) da el resto de una división print(f"El número {numero} es par.") else: print(f"El número {numero}
es impar.")

print("Proceso de paridad completado.") ``
```

**¿Qué sucede aquí?** 1. numero es 15. 2. La condición numero % 2 == 0 se evalúa: 15 % 2 es 1. Entonces, 1 == 0 es False. 3. Como la condición del if es False, el bloque else se ejecuta, imprimiendo "El número 15 es impar.". 4. Finalmente, se imprime "Proceso de paridad completado.".

Si numero fuera 10, la condición 10 % 2 == 0 sería True, y se ejecutaría el bloque if, imprimiendo "El número 10 es par.".

---

## 3. La Sentencia if/elif/else (Si... Sino Si... Sino...)

---

Cuando tenemos múltiples condiciones que deben evaluarse en un orden específico, y solo una de ellas debe activar un bloque de código, utilizamos elif (abreviatura de "else if"). Esto nos permite encadenar varias comprobaciones.

### Sintaxis:

```
python if primera_condicion: # Código si la primera_condicion es Verdadera elif segunda_condicion: #
Código si la primera_condicion fue Falsa, Y la segunda_condicion es Verdadera elif tercera_condicion: #
Código si las condiciones anteriores fueron Falsas, Y la tercera_condicion es Verdadera else: # Código si
NINGUNA de las condiciones anteriores fue Verdadera
```

**Puntos Clave:** \* Python evalúa las condiciones de arriba hacia abajo. Tan pronto como encuentra una condición True, ejecuta su bloque de código asociado y **salta el resto de la estructura elif/else**. \* Puedes tener tantos elif como necesites. \* El bloque else es opcional, pero es una buena práctica incluirlo para manejar cualquier caso no cubierto por las condiciones if o elif.

### Ejemplo Práctico:

Clasifiquemos la nota de un estudiante en un sistema de calificaciones.

```
```python nota = 78
```

```
if nota >= 90: print("Calificación: A (Excelente)") elif nota >= 80: print("Calificación: B (Notable)") elif nota >= 70: print("Calificación: C (Bueno)")
elif nota >= 60: print("Calificación: D (Suficiente)") else: print("Calificación: F (Insuficiente)")
```

```
print("Evaluación de nota completada.")```
```

Análisis del Flujo (con nota = 78): 1. if nota >= 90: 78 >= 90 es False. 2. elif nota >= 80: 78 >= 80 es False. 3. elif nota >= 70: 78 >= 70 es True. **¡Condición cumplida!** 4. Se ejecuta print("Calificación: C (Bueno)"). 5. El resto de la estructura (elif nota >= 60 y else) se **salta**. 6. Se imprime "Evaluación de nota completada".

Si la nota fuera 95, se ejecutaría el primer if y el resto se saltaría. Si fuera 55, todas las condiciones if/elif serían False, y se ejecutaría el else.

4. El Operador Condicional Ternario (Compacto if/else)

Python ofrece una forma más compacta de expresar una sentencia if/else simple, especialmente útil cuando el resultado de la condición se usa para asignar un valor a una variable. Esto se conoce como el **operador condicional ternario**.

Sintaxis:

```
python variable = valor_si_verdadero if condicion else valor_si_falso
```

Puntos Clave: * Se evalúa la condicion. * Si la condicion es True, variable toma el valor_si_verdadero. * Si la condicion es False, variable toma el valor_si_falso. * Es una forma de una sola línea para un if/else que asigna un valor.

Ejemplo Práctico:

Volvamos al ejemplo de paridad, pero de forma más concisa.

```
```python num = 7 paridad = "par" if (num % 2 == 0) else "impar" print(f"El número {num} es {paridad}.") # Salida: El número 7 es impar.
```

```
num = 10 paridad = "par" if (num % 2 == 0) else "impar" print(f"El número {num} es {paridad}.") # Salida: El número 10 es par.```
```

Este operador es muy útil para hacer el código más legible y compacto en situaciones donde la lógica condicional es sencilla y el objetivo es una asignación.

---

## 5. Consideraciones Importantes

- **Indentación:** Es la piedra angular de la sintaxis de Python. Una indentación incorrecta resultará en un IndentationError. Asegúrate de que todos los bloques de código dentro de un if, elif o else tengan el mismo nivel de indentación.

**Contexto Booleano (Truthiness):** En Python, no solo los valores True y False se consideran booleanos. Muchos otros valores tienen un "valor booleano" implícito:

- **Falsy (evalúan a False):** None, 0 (cero numérico), 0.0, "" (cadena vacía), [] (lista vacía), {} (diccionario vacío), () (tupla vacía), set() (conjunto vacío).
- **Truthy (evalúan a True):** Cualquier otro valor (números no cero, cadenas no vacías, listas no vacías, etc.).  
```python milista = [] if milista: # Esto es False porque la lista está vacía print("La lista no está vacía.") else: print("La lista está vacía.")

```
nombre = "Alice" if nombre: # Esto es True porque la cadena no está vacía print(f"Hola, {nombre}!") ```
```

- **Ausencia de switch:** A diferencia de otros lenguajes de programación (como C++, Java o JavaScript), Python no tiene una sentencia switch o case nativa. Para emular su funcionalidad, se suelen utilizar cadenas if/elif/else o, para casos más complejos y eficientes, diccionarios donde las claves son las condiciones y los valores son las funciones o acciones a ejecutar.

Conclusión

Las sentencias condicionales (`if`, `elif`, `else`) son herramientas esenciales que transforman un programa lineal en uno dinámico y adaptable. Nos permiten construir lógica compleja, reaccionar a diferentes entradas y guiar el flujo de ejecución de nuestros programas de manera inteligente. Dominar estas estructuras es un paso fundamental para escribir código más robusto y funcional. Te animo a practicar con diferentes escenarios y a experimentar con las condiciones y los bloques de código para afianzar tu comprensión. En la próxima lección, exploraremos otra herramienta crucial para el control de flujo: los bucles, que nos permitirán repetir bloques de código. ¡Sigue adelante con tu aprendizaje!

1.2 Lección 2.2: Bucles (for y while)

¡Excelente! Bienvenidos a esta nueva lección. Como vuestro profesor, me complace guiaros a través de uno de los pilares fundamentales de la programación: los bucles. Estas estructuras nos permiten automatizar tareas y procesar datos de manera eficiente, evitando la repetición de código y haciendo nuestros programas mucho más potentes y elegantes.

Lección 2.2: Bucles (for y while)

Introducción a los Bucles

Imaginad que necesitáis realizar la misma operación diez, cien o mil veces. ¿Escribiríais el mismo código una y otra vez? ¡Por supuesto que no! Para eso existen los **bucles**. Los bucles son estructuras de control que nos permiten ejecutar un bloque de código repetidamente, ya sea un número determinado de veces o mientras se cumpla una condición específica. En Python, las dos estructuras de bucle principales son `while` y `for`.

1. El Bucle `while`

El bucle `while` (mientras) ejecuta un bloque de código **mientras** una condición sea verdadera. La clave aquí es que la condición se evalúa **antes** de cada iteración. Si la condición es `False` desde el principio, el bloque de código dentro del `while` nunca se ejecutará.

Sintaxis Básica

```
```python while condicion_verdadera: # Bloque de código a repetir # (Asegúrate de que algo dentro del bucle cambie la condición # para evitar un bucle infinito) ```
```

#### Ejemplo Sencillo: Contador

Vamos a ver un ejemplo clásico para entender su funcionamiento: 

```
```python print("--- Inicio del bucle while ---") contador = 0 while contador < 5: print(f"El contador es: {contador}") contador += 1 # Incrementamos el contador en 1 en cada iteración print("--- Fin del bucle while ---") ```
```

****Explicación:**** 1. `contador` se inicializa en `0`. 2. La condición `contador < 5` es `True` (0 es menor que 5). 3. Se imprime el valor del contador. 4. `contador` se incrementa a `1`. 5. El bucle vuelve a evaluar la condición. Esto se repite hasta que `contador` llega a `5`. 6. Cuando `contador` es `5`, la condición `5 < 5` es `False`, y el bucle termina.

Palabras Clave en Bucles `while`: `break`

A veces, necesitamos salir de un bucle `while` antes de que su condición principal se vuelva `False`. Para esto, utilizamos la palabra clave `break`. `break` termina el bucle **inmediatamente** y la ejecución del programa continúa con la instrucción que sigue al bucle. ****Ejemplo:** Salir con una palabra clave Este es un ejemplo muy práctico que nos habéis proporcionado y que ilustra perfectamente el uso de `break`:

```
```python print("--- Programa de eco interactivo ---") salir = False while not salir: entrada = input("Di algo (o escribe 'adios' para salir): ") # Usamos input() para Python 3 if entrada == "adios": salir = True # Cambiamos la condición para que el bucle termine en la próxima evaluación # Alternativamente, podríamos usar 'break' aquí directamente: # break else: print(f"Tú dijiste: {entrada}") print("¡Hasta luego! El programa ha terminado.") ```
```

**\*\*Explicación:\*\*** 1. El bucle `while not salir` se ejecuta indefinidamente mientras `salir` sea `False`. 2. El usuario introduce texto. 3. Si la `entrada` es "adios", la variable `salir` se establece en `True`. En la siguiente evaluación de la condición `not salir`, esta será `False`, y el bucle terminará. 4. Si la `entrada` no es "adios", se imprime lo que el usuario escribió. Si hubiéramos usado `break` directamente dentro del

`if`, el bucle habría terminado en el mismo instante en que se detecta "adios", sin necesidad de esperar a la siguiente evaluación de la condición del `while`. Ambas formas son válidas, pero `break` es más directo para una salida inmediata.

### Palabras Clave en Bucles while: continue

Otra palabra clave útil dentro de los bucles es `continue`. Como habréis adivinado, `continue` no hace otra cosa que pasar directamente a la \*siguiente iteración\* del bucle, saltándose el resto del código que haya en la iteración actual. **Ejemplo: Felicitar solo en edades impares** Este ejemplo, también extraído de vuestro material, es una pequeña modificación de un programa de felicitaciones: 

```
python print("--- Programa de felicitaciones (edades impares) ---") edad = 0 while edad < 18: edad = edad + 1 # Incrementamos la edad if edad % 2 == 0: # Si la edad es par... continue # ...saltamos a la siguiente iteración print("Felicidades, tienes " + str(edad) + " años (edad impar).") print("--- Fin del programa de felicitaciones ---")
```

**Explicación:** 1. El bucle se ejecuta mientras `edad` sea menor de `18`. 2. En cada iteración, `edad` se incrementa. 3. Se comprueba si `edad` es par (`edad % 2 == 0`). 4. Si es par, `continue` hace que el programa salte la línea `print(...)` y pase directamente a la siguiente iteración del bucle (es decir, vuelve a evaluar `while edad < 18`). 5. Si es impar, el `if` no se cumple, `continue` no se ejecuta, y se imprime el mensaje de felicitación. Como veis, con esta modificación el programa sólo imprimiría felicitaciones cuando la edad fuera impar.

## 2. El Bucle for . . . in

A los que hayáis tenido experiencia previa con según qué lenguajes, este bucle os va a sorprender gratamente. En Python, `for` se utiliza como una forma genérica de **iterar sobre una secuencia**. Y como tal, intenta facilitar su uso para este fin. Un bucle `for` es ideal cuando sabemos de antemano sobre qué colección de elementos queremos iterar, o cuántas veces queremos repetir una acción.

### Sintaxis Básica

```
python for elemento in secuencia: # Bloque de código a ejecutar para cada elemento
```

#### Ejemplo: Iterar sobre una lista

Este es el aspecto de un bucle `for` en Python, utilizando el ejemplo que nos habéis proporcionado: 

```
python print("--- Iterando sobre una lista ---") secuencia = ["uno", "dos", "tres"] for elemento in secuencia: print(elemento) print("--- Fin de la iteración ---")
```

**Explicación:** 1. La variable `secuencia` es una lista con tres cadenas de texto. 2. El bucle `for` toma cada `elemento` de la `secuencia` en orden. 3. En la primera iteración, `elemento` es "uno", y se imprime. 4. En la segunda, `elemento` es "dos", y se imprime. 5. En la tercera, `elemento` es "tres", y se imprime. 6. Una vez que no quedan más elementos en la `secuencia`, el bucle `for` termina.

#### Ejemplos Adicionales de for . . . in

El bucle `for` es increíblemente versátil. Puede iterar sobre cualquier objeto **iterable** en Python, como listas, tuplas, cadenas de texto, diccionarios, y rangos numéricos. **Iterar sobre una cadena de texto:**

```
python print("\n--- Iterando sobre una cadena de texto ---") saludo = "Hola Mundo" for letra in saludo: print(letra)
```

**Iterar con la función `range`:** La función `range()` es muy útil para generar secuencias de números, lo que a menudo se usa con los bucles `for` cuando queremos repetir una acción un número específico de veces.

- `range(n)`: Genera números desde 0 hasta n-1. 

```
python print("\n--- Iterando con range(5) ---") for i in range(5): # Genera 0, 1, 2, 3, 4 print(f"Iteración número: {i}")
```
- `range(inicio, fin)`: Genera números desde inicio hasta fin-1. 

```
python print("\n--- Iterando con range(2, 7) ---") for num in range(2, 7): # Genera 2, 3, 4, 5, 6 print(f"Número: {num}")
```
- `range(inicio, fin, paso)`: Genera números desde inicio hasta fin-1, incrementando en paso. 

```
python print("\n--- Iterando con range(10, 0, -2) (cuenta atrás) ---") for j in range(10, 0, -2): # Genera 10, 8, 6, 4, 2 print(f"Cuenta atrás: {j}")
```

### ¿Cuándo usar for y cuándo while?

La elección entre un bucle `for` y un bucle `while` depende de la naturaleza de la tarea:

#### Usa for cuando:

- Sabes de antemano cuántas veces quieres iterar (por ejemplo, 5 veces).
- Necesitas procesar cada elemento de una colección (lista, tupla, cadena, etc.).
- La iteración se basa en una secuencia finita.

#### Usa while cuando:

- La repetición depende de una condición que puede cambiar durante la ejecución del bucle.
- No sabes de antemano cuántas iteraciones serán necesarias (por ejemplo, esperar la entrada de un usuario hasta que escriba "salir").
- Necesitas un bucle que se ejecute indefinidamente hasta que se cumpla una condición interna (a menudo con break).

## Conclusión

Los bucles `for` y `while` son herramientas esenciales en la caja de herramientas de cualquier programador. Nos permiten escribir código más conciso, eficiente y potente al automatizar tareas repetitivas. Recordad que `break` os permite salir de un bucle prematuramente, y `continue` os permite saltar a la siguiente iteración. Dominar su uso os abrirá un mundo de posibilidades en la programación.

Espero que esta lección os haya sido útil para comprender a fondo los bucles en Python. ¡Ahora, a practicar!

---

## Ejercicio Práctico

Para consolidar lo aprendido, os propongo los siguientes ejercicios:

**Adivina el número (usando `while` y `break`):** Escribe un programa que genere un número aleatorio entre 1 y 10. Luego, pide al usuario que adivine el número. El programa debe seguir pidiendo números hasta que el usuario adivine correctamente. Si el usuario escribe "salir", el programa debe terminar.

```
```python import random numero_secreto = random.randint(1, 10) print("Adivina el número secreto (entre 1 y 10). Escribe 'salir' para terminar.")
```

Tu código aquí

...

Suma de elementos pares (usando `for` y `continue`): Dada una lista de números, usa un bucle `for` para calcular la suma de todos los números pares en la lista. Utiliza `continue` para saltar los números impares.

```
```python numeros = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] suma_pares = 0 print("\nCalculando la suma de números pares:")
```

### Tu código aquí

---

...

¡Mucha suerte con los ejercicios!

## 1.3 Lección 2.3: Funciones: Definición, Parámetros y Retorno

¡Bienvenidos a la Lección 2.3 de nuestro curso!

---

## Lección 2.3: Funciones: Definición, Parámetros y Retorno

---

### Introducción: El Poder de la Modularización

---

En programación, a medida que nuestros proyectos crecen, el código puede volverse complejo y difícil de manejar. Aquí es donde las funciones entran en juego como una de las herramientas más fundamentales y poderosas. Las funciones nos permiten organizar nuestro código en bloques lógicos y reutilizables, mejorando la legibilidad, la mantenibilidad y la eficiencia de nuestras aplicaciones.

En esta lección, exploraremos qué son las funciones, cómo definir las, cómo hacer que acepten información a través de **parámetros** y cómo pueden devolver resultados utilizando la instrucción `return`.

---

### 1. ¿Qué es una Función?

---

Imagina una función como una "mini-programa" o una "receta" dentro de tu programa principal. Es un bloque de código con un nombre específico que realiza una tarea particular. Una vez definida, puedes "llamar" o "invocar" esta función tantas veces como necesites, sin tener que reescribir el mismo código repetidamente.

**Propósitos principales de las funciones:**

- **Modularización:** Dividir un problema grande en problemas más pequeños y manejables.
  - **Reutilización de código:** Escribir un bloque de código una vez y usarlo en múltiples lugares.
  - **Legibilidad:** Hacer que el código sea más fácil de entender.
  - **Mantenibilidad:** Facilitar la depuración y modificación del código.
- 

## 2. Definición de una Función: La Sintaxis def

---

Para crear una función en Python, utilizamos la palabra clave `def`, seguida del nombre de la función, un par de paréntesis `()` y dos puntos `:`. El **cuerpo de la función** (el código que se ejecutará cuando la función sea llamada) debe estar indentado.

**Sintaxis básica:**

```
```python def nombredela_funcion(): # Cuerpo de la función # Este es el bloque de código que conforma el código a ejecutar por la función. print("¡Hola desde mi primera función!") print("Esta función no recibe parámetros ni devuelve valores.")```
```

Para ejecutar la función, simplemente la "llamamos" por su nombre:

```
nombredelafuncion() nombredelafuncion() # Podemos llamarla varias veces ```
```

Explicación:

- `def`: Palabra clave para definir una función.
 - `nombre_de_la_funcion`: Un identificador único para tu función. Sigue las mismas reglas que los nombres de variables (minúsculas, guiones bajos para separar palabras).
 - `()`: Paréntesis que contendrán los parámetros (si los hay).
 - `:`: Indica el comienzo del cuerpo de la función.
 - **Indentation**: Todo el código dentro de la función debe estar indentado (generalmente con 4 espacios).
-

3. Parámetros: La Entrada de Nuestras Funciones

Las funciones son mucho más útiles cuando pueden operar con datos específicos que les proporcionamos. Estos datos se conocen como **parámetros** (cuando se definen en la función) o **argumentos** (cuando se pasan al llamar la función). Los parámetros actúan como variables locales dentro de la función, cuyos valores son asignados cuando la función es invocada.

Sintaxis con parámetros:

```
```python def saludar_persona(nombre): """Esta función saluda a una persona por su nombre.""" print(f"¡Hola, {nombre}!")```
```

**Llamando a la función y pasando argumentos:**

---

```
saludarpersona("Alice") # "Alice" es el argumento para el parámetro 'nombre' saludarpersona("Bob") ```
```

Podemos definir múltiples parámetros, separándolos con comas:

```
```python def mostrar_info(nombre, edad): """ Esta función imprime el nombre y la edad pasados como parámetros. """ print(f"Nombre: {nombre}") print(f"Edad: {edad}")```
```

Llamadas a la función con múltiples argumentos:

```
mostrarinfo("Carlos", 30) mostrarinfo("Ana", 25) ```
```

Nota: El orden de los argumentos al llamar la función debe coincidir con el orden de los parámetros definidos en la función, a menos que uses argumentos por palabra clave.

4. Retorno de Valores: La Salida de Nuestras Funciones

Hasta ahora, nuestras funciones solo han "hecho" cosas (como imprimir en la consola). Sin embargo, muchas veces queremos que una función calcule algo y nos **devuelva** un resultado para que podamos usarlo en otras partes de nuestro programa. Para esto, utilizamos la palabra clave `return`.

Cuando una función ejecuta una instrucción `return`, finaliza su ejecución y el valor especificado después de `return` es enviado de vuelta al lugar donde la función fue llamada.

Sintaxis con `return`:

```
```python def sumar(a, b): """ Esta función suma dos números y devuelve el resultado. """ resultado = a + b return resultado # La función devuelve el valor de 'resultado'```
```

### Llamando a la función y almacenando su valor de retorno:

---

```
total = sumar(5, 3) print(f"La suma es: {total}") # Output: La suma es: 8
```

### Podemos usar el valor de retorno directamente:

---

```
print(f"La suma de 10 y 20 es: {sumar(10, 20)}") # Output: La suma de 10 y 20 es: 30
```

### Diferencia clave entre `print` y `return`:

---

```
def funcionconprint(x, y): print(x * y) # Solo muestra en consola, no devuelve un valor
```

```
def funcionconreturn(x, y): return x * y # Devuelve el valor, que puede ser usado
```

```
valorimpreso = funcionconprint(2, 4) print(f"Valor de funcionconprint: {valorimpreso}") # Output: 8 (de la función), luego Valor de funcionconprint: None
```

```
valorretornado = funcionconreturn(2, 4) print(f"Valor de funcionconreturn: {valorretornado}") # Output: Valor de funcionconreturn: 8 ```
```

**Importante:** Una función puede devolver cualquier tipo de dato (números, cadenas, listas, objetos, etc.). Si una función no tiene una instrucción `return` explícita, implícitamente devuelve `None`.

---

## 5. Docstrings: Documentando Nuestras Funciones

---

Como se mencionó en el contexto, una buena práctica en Python es incluir una cadena de texto como primera línea del cuerpo de la función. Estas cadenas se conocen con el nombre de **docstring** (cadena de documentación) y sirven, como su nombre indica, a modo de documentación de la función.

Los docstrings se escriben entre comillas triples ("""Docstring aquí""") y son increíblemente útiles para explicar qué hace la función, sus parámetros y qué devuelve.

#### Ejemplo con Docstring:

```
```python def mifuncioncon_docstring(param1, param2): """ Esta función imprime los dos valores pasados como parámetros. Es un ejemplo de cómo documentar una función usando un docstring.```
```

Parámetros:

param1 (cualquier tipo): El primer valor a imprimir.

param2 (cualquier tipo): El segundo valor a imprimir.

```
"""
```

```
print(f"Primer parámetro: {param1}")
```

```
print(f"Segundo parámetro: {param2}")
```

Los docstrings son accesibles a través de la función `help()` o el operador `?` en IPython:

```
help(mifuncioncon_docstring) ```
```

Salida esperada de `help(mi_funcion_con_docstring)`:

```
''' Help on function mifuncioncon_docstring in module main:
```

```
mifuncioncon_docstring(param1, param2) Esta función imprime los dos valores pasados como parámetros. Es un ejemplo de cómo documentar una función usando un docstring.
```

```
Parámetros:
```

```
param1 (cualquier tipo): El primer valor a imprimir.
```

```
param2 (cualquier tipo): El segundo valor a imprimir.
```

```
'''
```

Esto es lo que imprime el operador `?` de iPython o la función `help()` de Python, haciendo que tu código sea auto-documentado y mucho más fácil de entender para ti y para otros desarrolladores.

6. Beneficios Clave de Usar Funciones

- **Modularización:** Divide el código en unidades lógicas y manejables.
- **Reutilización:** Evita la repetición de código (principio DRY: Don't Repeat Yourself).
- **Legibilidad:** El código es más fácil de leer y entender, ya que cada función tiene un propósito claro.
- **Mantenibilidad:** Los cambios o correcciones en una parte del código solo necesitan hacerse en la función correspondiente.
- **Depuración:** Es más fácil encontrar y corregir errores en funciones pequeñas y aisladas.

Conclusión

Las funciones son la columna vertebral de la programación estructurada. Al dominar su definición, el uso de parámetros para la entrada y la instrucción `return` para la salida, estarás en camino de escribir código más limpio, eficiente y profesional. Recuerda siempre documentar tus funciones con docstrings para hacerlas aún más útiles.

Ejercicios Prácticos

Crea una función `calcular_area_rectangulo`:

- Debe aceptar dos parámetros: ancho y alto.
- Debe calcular el área del rectángulo (`ancho * alto`).
- Debe devolver el resultado.
- Llama a la función con diferentes valores y muestra el área.

Crea una función `saludar_idioma`:

- Debe aceptar dos parámetros: nombre y idioma (por ejemplo, "es" para español, "en" para inglés).
- Si idioma es "es", debe imprimir "¡Hola, [nombre]!".
- Si idioma es "en", debe imprimir "Hello, [nombre]!".
- Si el idioma no es reconocido, debe imprimir "Idioma no soportado.".
- Asegúrate de incluir un docstring claro para tu función.

Módulo 2: Módulo 3: Estructuras de Datos Avanzadas y Manejo de Archivos

2.1 Lección 3.1: Listas, Tuplas y Conjuntos

¡Excelente! Bienvenidos a la Lección 3.1 de nuestro curso de programación. En el capítulo anterior, exploramos los tipos de datos fundamentales como números, cadenas de texto y booleanos, que son los bloques de construcción individuales de la información. Hoy, daremos un paso adelante para aprender a *organizar* y *gestionar* múltiples piezas de datos de manera eficiente.

Esta lección se centrará en las **colecciones de datos**, estructuras que nos permiten agrupar valores. Específicamente, profundizaremos en las **Listas**, **Tuplas** y **Conjuntos**, entendiendo sus características distintivas, cómo crearlas, cómo manipular sus elementos y cuándo elegir una

sobre otra.

Lección 3.1: Listas, Tuplas y Conjuntos

1. Introducción a las Colecciones de Datos

En la programación, rara vez trabajamos con un único dato aislado. A menudo, necesitamos manejar grupos de elementos relacionados: una lista de estudiantes, una serie de temperaturas, un conjunto de opciones únicas. Para esto, Python nos ofrece las **colecciones**.

Las colecciones son tipos de datos que pueden almacenar múltiples valores. Las principales diferencias entre ellas radican en tres aspectos clave:

1. **Orden:** ¿Los elementos mantienen un orden específico? ¿Podemos acceder a ellos por su posición (índice)?
2. **Mutabilidad:** ¿Podemos modificar la colección (añadir, eliminar o cambiar elementos) después de haberla creado?
3. **Permite Duplicados:** ¿La colección permite que haya elementos repetidos?

Hoy exploraremos tres de las colecciones más comunes y versátiles: Listas, Tuplas y Conjuntos.

2. Listas: Colecciones Ordenadas y Mutables

Las listas son, quizás, la colección más utilizada en Python. Son increíblemente flexibles y potentes.

2.1. ¿Qué son las Listas?

Una **lista** es una colección **ordenada** y **mutable** de elementos. Esto significa dos cosas fundamentales:

- **Ordenada:** Los elementos tienen una posición definida (un índice), y ese orden se mantiene. Podemos acceder a los elementos por su índice.
- **Mutable:** Podemos cambiar, añadir o eliminar elementos de la lista *después* de que haya sido creada.

Las listas son el equivalente a lo que en otros lenguajes de programación se conoce como *arrays* o *vectores*. Pueden contener cualquier tipo de dato: números, cadenas, booleanos, e incluso otras listas.

2.2. Creación de Listas

Para crear una lista, simplemente encerramos los elementos entre **corchetes** `[]`, separados por comas.

```
```python
```

#### Una lista simple de números

---

```
numeros = [10, 20, 30, 40, 50] print(f"Lista de números: {numeros}") # Salida: Lista de números: [10, 20, 30, 40, 50]
```

#### Una lista con diferentes tipos de datos (heterogénea)

---

```
miListaheterogenea = [22, True, "una lista", 3.14, False] print(f"Lista heterogénea: {miListaheterogenea}") # Salida: Lista heterogénea: [22, True, 'una lista', 3.14, False]
```

#### Una lista que contiene otra lista (lista anidada)

---

```
listaanidada = [1, "dos", [3, 4], True] print(f"Lista anidada: {listaanidada}") # Salida: Lista anidada: [1, 'dos', [3, 4], True]
```

#### Una lista vacía

---

```
listavacia = [] print(f"Lista vacía: {listavacia}") # Salida: Lista vacía: [] ```
```

### 2.3. Acceso a Elementos (Indexación y Slicing)

Dado que las listas son ordenadas, podemos acceder a sus elementos utilizando su **índice**. Es crucial recordar que en Python (y en la mayoría de los lenguajes de programación), el índice del primer elemento es **0**, no 1.

```
python mi_lista = ["manzana", "banana", "cereza", "dátil"]
```

#### Acceder al primer elemento (índice 0)

---

```
primerelemento = mi_lista[0] print(f"Primer elemento: {primerelemento}") # Salida: Primer elemento: manzana
```

#### Acceder al tercer elemento (índice 2)

---

```
tercerelemento = mi_lista[2] print(f"Tercer elemento: {tercerelemento}") # Salida: Tercer elemento: cereza
```

#### Acceso con índices negativos:

---

**-1 se refiere al último elemento, -2 al penúltimo, y así sucesivamente.**

---

```
ultimoelemento = mi_lista[-1] print(f"Último elemento: {ultimoelemento}") # Salida: Último elemento: dátil
```

#### Acceder a un elemento de una lista anidada

---

```
listaanidada = [10, 20, ["a", "b", "c"], 40] elementoanidado = listaanidada[2][1] # Accede a la lista en el índice 2, luego al elemento en el índice 1 de esa lista. print(f"Elemento de lista anidada: {elementoanidado}") # Salida: Elemento de lista anidada: b
```

#### Slicing (Rebanado)

Podemos extraer sub-listas utilizando la técnica de *slicing*. La sintaxis es `[inicio:fin:pasos]`.

- **inicio**: Índice donde comienza la rebanada (incluido). Si se omite, es 0.
- **fin**: Índice donde termina la rebanada (excluido). Si se omite, es el final de la lista.
- **pasos**: Incremento entre elementos (por defecto es 1).

```
python numeros = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

#### Elementos del índice 2 al 5 (excluido)

---

```
sublista1 = numeros[2:5] print(f"numeros[2:5]: {sublista1}") # Salida: numeros[2:5]: [2, 3, 4]
```

#### Desde el principio hasta el índice 4 (excluido)

---

```
sublista2 = numeros[:4] print(f"numeros[:4]: {sublista2}") # Salida: numeros[:4]: [0, 1, 2, 3]
```

#### Desde el índice 6 hasta el final

---

```
sublista3 = numeros[6:] print(f"numeros[6:]: {sublista3}") # Salida: numeros[6:]: [6, 7, 8, 9]
```

#### Copia completa de la lista

---

```
copialista = numeros[:] print(f"numeros[:]: {copialista}") # Salida: numeros[:]: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

## Con paso: elementos pares

---

```
pares = numeros[::2] print(f'numeros[::2]: {pares}') # Salida: numeros[::2]: [0, 2, 4, 6, 8]
```

## Con paso: lista invertida

---

```
invertida = numeros[::-1] print(f'numeros[::-1]: {invertida}') # Salida: numeros[::-1]: [9, 8, 7, 6, 5, 4, 3, 2, 1, 0] ``
```

### 2.4. Métodos Principales de Listas (Mutabilidad en Acción)

La mutabilidad es una de las características más poderosas de las listas. Podemos modificar su contenido de varias maneras:

**Modificar un elemento:** python `frutas = ["manzana", "banana", "cereza"] frutas[1] = "kiwi" print(f"Lista modificada: {frutas}")` # Salida: Lista modificada: ['manzana', 'kiwi', 'cereza']

**append(elemento):** Añade un elemento al final de la lista. python `frutas.append("naranja") print(f"Después de append: {frutas}")` # Salida: Después de append: ['manzana', 'kiwi', 'cereza', 'naranja']

**insert(indice, elemento):** Inserta un elemento en una posición específica. python `frutas.insert(1, "uva")` # Inserta "uva" en el índice 1 `print(f"Después de insert: {frutas}")` # Salida: Después de insert: ['manzana', 'uva', 'kiwi', 'cereza', 'naranja']

**remove(elemento):** Elimina la primera ocurrencia de un elemento específico. Si el elemento no existe, produce un error `ValueError`. python `frutas.remove("kiwi") print(f"Después de remove: {frutas}")` # Salida: Después de remove: ['manzana', 'uva', 'cereza', 'naranja']

**pop(indice):** Elimina y devuelve el elemento en el índice especificado. Si no se especifica un índice, elimina y devuelve el último elemento. ``python `elementoeliminado = frutas.pop(0)` # Elimina "manzana" `print(f"Elemento eliminado con pop: {elementoeliminado}")` # Salida: Elemento eliminado con pop: manzana `print(f"Después de pop(0): {frutas}")` # Salida: Después de pop(0): ['uva', 'cereza', 'naranja']

`ultimoelemento = frutas.pop()` # Elimina "naranja" `print(f"Último elemento eliminado con pop(): {ultimoelemento}")` # Salida: Último elemento eliminado con pop(): naranja `print(f"Después de pop(): {frutas}")` # Salida: Después de pop(): ['uva', 'cereza'] ``

**del (sentencia):** Elimina elementos por índice o rebanada, o la lista completa. ``python `del frutas[0]` # Elimina "uva" `print(f"Después de del frutas[0]: {frutas}")` # Salida: Después de del frutas[0]: ['cereza']

### del frutas # Esto eliminaría la variable 'frutas' completamente

---

``

**extend(otra\_lista):** Añade todos los elementos de otra lista al final de la lista actual. python `lista_a = [1, 2] lista_b = [3, 4] lista_a.extend(lista_b) print(f"Después de extend: {lista_a}")` # Salida: Después de extend: [1, 2, 3, 4] (Nota: + también concatena listas, pero crea una nueva lista en lugar de modificar la existente.)

**sort():** Ordena los elementos de la lista en su lugar (modifica la lista original). ``python `numeros = [3, 1, 4, 1, 5, 9, 2] numeros.sort()` `print(f"Lista ordenada: {numeros}")` # Salida: Lista ordenada: [1, 1, 2, 3, 4, 5, 9]

### Para ordenar de forma descendente:

---

`numeros.sort(reverse=True) print(f"Lista ordenada descendente: {numeros}")` # Salida: Lista ordenada descendente: [9, 5, 4, 3, 2, 1, 1] ``  
\* (Nota: `sorted(lista)` devuelve una nueva lista ordenada sin modificar la original.)\*

**reverse():** Invierte el orden de los elementos en la lista en su lugar. python `letras = ['a', 'b', 'c'] letras.reverse()` `print(f"Lista invertida: {letras}")` # Salida: Lista invertida: ['c', 'b', 'a']

**count(elemento):** Devuelve el número de veces que un elemento aparece en la lista. python `mi_lista = [1, 2, 2, 3, 2, 4] conteo_dos = mi_lista.count(2) print(f"El número 2 aparece {conteo_dos} veces.")` # Salida: El número 2 aparece 3 veces.

**index(elemento):** Devuelve el índice de la primera ocurrencia de un elemento. Si el elemento no existe, produce un error `ValueError`. python `mi_lista = ["rojo", "verde", "azul"] indice_verde = mi_lista.index("verde") print(f"El índice de`

```
'verde' es: {indice_verde}") # Salida: El índice de 'verde' es: 1
```

## 2.5. Comprensión de Listas (List Comprehensions)

La comprensión de listas es una característica muy potente y elegante de Python, tomada de lenguajes de programación funcional como Haskell. Permite crear listas de forma concisa y legible a partir de otras listas o iterables.

Su sintaxis general es: `[expresion for elemento in iterable if condicion]`

- **expresion:** Lo que se va a añadir a la nueva lista.
- **elemento:** Variable que toma cada valor del iterable.
- **iterable:** La colección original sobre la que se itera.
- **condicion (opcional):** Filtro para incluir solo ciertos elementos.

Este enfoque se desaconseja en favor de las comprensiones de listas para muchas operaciones que antes se hacían con `map()` o `filter()`.

```
```python
```

Ejemplo: Elevar al cuadrado todos los elementos de una lista

```
numeros = [1, 2, 3, 4, 5] cuadrados = [n * n for n in numeros] print(f"Cuadrados: {cuadrados}") # Salida: Cuadrados: [1, 4, 9, 16, 25]
```

Ejemplo con una condición (solo números pares al cuadrado)

```
parescuadrados = [n * n for n in numeros if n % 2 == 0] print(f"Pares al cuadrado: {parescuadrados}") # Salida: Pares al cuadrado: [4, 16]
```

Ejemplo: Crear una lista de cadenas en mayúsculas

```
palabras = ["hola", "mundo", "python"] mayusculas = [palabra.upper() for palabra in palabras] print(f"Mayúsculas: {mayusculas}") # Salida: Mayúsculas: ['HOLA', 'MUNDO', 'PYTHON'] ```
```

3. Tuplas: Colecciones Ordenadas e Inmutables

Las tuplas son muy similares a las listas en algunos aspectos, pero tienen una diferencia crucial: son **inmutables**.

3.1. ¿Qué son las Tuplas?

Una **tupla** es una colección **ordenada** e **inmutable** de elementos.

- **Ordenada:** Al igual que las listas, los elementos tienen una posición definida (índice) y ese orden se mantiene.
- **Inmutable:** Una vez que una tupla ha sido creada, **no se puede modificar**. No se pueden añadir, eliminar o cambiar sus elementos.

3.2. Creación de Tuplas

Para crear una tupla, encerramos los elementos entre **paréntesis ()**, separados por comas.

```
```python
```

### Una tupla simple de números

---

```
coordenadas = (10, 20) print(f"Tupla de coordenadas: {coordenadas}") # Salida: Tupla de coordenadas: (10, 20)
```

### Una tupla heterogénea

---

```
mituplaheterogenea = (1, "dos", True, 3.14) print(f"Tupla heterogénea: {mituplaheterogenea}") # Salida: Tupla heterogénea: (1, 'dos', True, 3.14)
```

## Tupla con un solo elemento (¡nota la coma!)

---

Es crucial añadir una coma después del elemento para que Python lo interprete como una tupla.

---

```
tuplaunelemento = (5,) print(f"Tuple de un elemento: {tuplaunelemento}, tipo: {type(tuplaunelemento)}") # Salida: Tuple de un elemento: (5,), tipo:
```

Sin la coma, Python lo interpreta como un entero entre paréntesis.

---

```
noestupla = (5) print(f"No es tupla: {noestupla}, tipo: {type(noestupla)}") # Salida: No es tupla: 5, tipo:
```

## Una tupla vacía

---

```
tuplavacia = () print(f"Tuple vacía: {tuplavacia}") # Salida: Tuple vacía: ()
```

### 3.3. Acceso a Elementos (Indexación y Slicing)

Al igual que las listas, las tuplas soportan indexación y slicing para acceder a sus elementos, ya que son ordenadas.

```
python dias_semana = ("Lunes", "Martes", "Miércoles", "Jueves", "Viernes")
```

## Acceder al primer elemento

---

```
primerdia = diasemana[0] print(f"Primer día: {primer_dia}") # Salida: Primer día: Lunes
```

## Slicing

---

```
finsemana = diasemana[3:] print(f"Fin de semana (laboral): {fin_semana}") # Salida: Fin de semana (laboral): ('Jueves', 'Viernes')
```

### 3.4. Inmutabilidad en Acción

La inmutabilidad significa que no podemos usar métodos que modifiquen la tupla.

```
python mi_tupla = (1, 2, 3)
```

## Intentar modificar un elemento (esto causará un error)

---

```
mi_tupla[0] = 10 # TypeError: 'tuple' object does not support item assignment
```

---

## Intentar añadir un elemento (esto causará un error)

---

```
mi_tupla.append(4) # AttributeError: 'tuple' object has no attribute 'append'
```

---

## Intentar eliminar un elemento (esto causará un error)

---

```
mi_tupla.remove(1) # AttributeError: 'tuple' object has no attribute 'remove'
```

---

Sin embargo, podemos reasignar la variable a una nueva tupla: `python mitupla = (1, 2, 3) mitupla = mitupla + (4, 5)` # Esto crea una *nueva* tupla y la asigna a la variable `mitupla` `print(f"Nueva tupla (reassignada): {mi_tupla}")` # Salida: Nueva tupla (reassignada): (1, 2,

3, 4, 5) ```

### 3.5. Métodos Principales de Tuplas

Debido a su inmutabilidad, las tuplas tienen menos métodos que las listas. Los más comunes son:

**count(elemento):** Devuelve el número de veces que un elemento aparece en la tupla. python numeros\_tupla = (1, 2, 2, 3, 2, 4) conteo\_dos\_tupla = numeros\_tupla.count(2) print(f"El número 2 aparece {conteo\_dos\_tupla} veces en la tupla.") # Salida: El número 2 aparece 3 veces en la tupla.

**index(elemento):** Devuelve el índice de la primera ocurrencia de un elemento. python colores\_tupla = ("rojo", "verde", "azul") indice\_verde\_tupla = colores\_tupla.index("verde") print(f"El índice de 'verde' en la tupla es: {indice\_verde\_tupla}") # Salida: El índice de 'verde' en la tupla es: 1

### 3.6. ¿Cuándo usar Tuplas?

Las tuplas son útiles cuando:

- Necesitas una colección de elementos que **no deben cambiar** a lo largo de la ejecución del programa (por ejemplo, constantes, configuraciones).
- Una función necesita devolver múltiples valores (se devuelven como una tupla).
- Se utilizan como claves en diccionarios (siempre que sus elementos sean inmutables).
- Para representar registros de datos donde la posición tiene un significado fijo (ej. coordenadas (x, y)).

## 4. Conjuntos: Colecciones Desordenadas sin Duplicados

---

Los conjuntos son una colección muy particular, inspirada en el concepto matemático de conjuntos.

### 4.1. ¿Qué son los Conjuntos?

Un **conjunto** es una colección **desordenada** de elementos **únicos**.

- **Desordenada:** Los elementos no tienen un orden definido y no se puede acceder a ellos por índice.
- **Sin Duplicados:** Un conjunto automáticamente elimina cualquier elemento duplicado. Cada elemento en un conjunto es único.
- **Mutable:** El conjunto en sí mismo es mutable; podemos añadir o eliminar elementos. Sin embargo, los *elementos individuales* que contiene deben ser inmutables (números, cadenas, tuplas, pero no listas o diccionarios).

### 4.2. Creación de Conjuntos

Para crear un conjunto, encerramos los elementos entre **llaves {}**, separados por comas. Para crear un conjunto vacío, debemos usar la función `set()`, ya que `{}` crea un diccionario vacío.

```python

Un conjunto de números (los duplicados se eliminan automáticamente)

```
numerosset = {1, 2, 3, 2, 4, 1} print(f"Conjunto de números: {numerosset}") # Salida: Conjunto de números: {1, 2, 3, 4} (el orden puede variar)
```

Un conjunto de cadenas

```
frutaset = {"manzana", "banana", "cereza", "manzana"} print(f"Conjunto de frutas: {frutaset}") # Salida: Conjunto de frutas: {'cereza', 'banana', 'manzana'} (el orden puede variar)
```

Crear un conjunto a partir de una lista (útil para eliminar duplicados)

```
listaconduplicados = [1, 1, 2, 3, 3, 4] conjuntosinduplicados = set(listaconduplicados) print(f"Conjunto sin duplicados: {conjuntosinduplicados}") # Salida: Conjunto sin duplicados: {1, 2, 3, 4}
```

Un conjunto vacío

```
conjuntovacio = set() print(f"Conjunto vacío: {conjuntovacio}, tipo: {type(conjuntovacio)}") # Salida: Conjunto vacío: set(), tipo:
```

¡Cuidado! {} crea un diccionario vacío, no un conjunto vacío.

```
diccionariovacio = {} print(f"Diccionario vacío: {diccionariovacio}, tipo: {type(diccionariovacio)}") # Salida: Diccionario vacío: {}, tipo: ``
```

4.3. Acceso a Elementos (No Indexación)

Dado que los conjuntos son desordenados, **no podemos acceder a sus elementos por índice**. Intentar hacerlo resultará en un error.

```
python mi_conjunto = {10, 20, 30}
```

print(mi_conjunto[0]) # TypeError: 'set' object is not subscriptable

...

Para verificar si un elemento está en un conjunto, usamos el operador in:

```
python print(f"¿Está 20 en mi_conjunto? {20 in mi_conjunto}") # Salida: ¿Está 20 en mi_conjunto? True
print(f"¿Está 50 en mi_conjunto? {50 in mi_conjunto}") # Salida: ¿Está 50 en mi_conjunto? False
```

4.4. Métodos Principales de Conjuntos (Operaciones de Conjuntos)

Los conjuntos son ideales para realizar operaciones matemáticas de conjuntos:

add(elemento): Añade un elemento al conjunto. Si el elemento ya existe, no hace nada. python colores = {"rojo", "verde"} colores.add("azul") print(f"Después de add 'azul': {colores}") # Salida: Después de add 'azul': {'verde', 'azul', 'rojo'} colores.add("rojo") # No se añade de nuevo print(f"Después de add 'rojo' (sin cambio): {colores}") # Salida: Después de add 'rojo' (sin cambio): {'verde', 'azul', 'rojo'}

remove(elemento): Elimina un elemento. Si el elemento no existe, produce un error KeyError. python colores.remove("verde") print(f"Después de remove 'verde': {colores}") # Salida: Después de remove 'verde': {'azul', 'rojo'}

colores.remove("amarillo") # KeyError: 'amarillo'

...

discard(elemento): Elimina un elemento. Si el elemento no existe, no hace nada (no produce error). python colores.discard("azul") print(f"Después de discard 'azul': {colores}") # Salida: Después de discard 'azul': {'rojo'} colores.discard("amarillo") # No hace nada, no hay error print(f"Después de discard 'amarillo' (sin cambio): {colores}") # Salida: Después de discard 'amarillo' (sin cambio): {'rojo'}

union(otro_conjunto) o |: Devuelve un nuevo conjunto con todos los elementos de ambos conjuntos. python set1 = {1, 2, 3} set2 = {3, 4, 5} union_sets = set1.union(set2) print(f"Unión: {union_sets}") # Salida: Unión: {1, 2, 3, 4, 5} print(f"Unión con |: {set1 | set2}") # Salida: Unión con |: {1, 2, 3, 4, 5}

intersection(otro_conjunto) o &: Devuelve un nuevo conjunto con los elementos comunes a ambos conjuntos. python interseccion_sets = set1.intersection(set2) print(f"Intersección: {interseccion_sets}") # Salida: Intersección: {3} print(f"Intersección con &: {set1 & set2}") # Salida: Intersección con &: {3}

difference(otro_conjunto) o -: Devuelve un nuevo conjunto con los elementos que están en el primer conjunto pero no en el segundo. python diferencia_sets = set1.difference(set2) print(f"Diferencia (set1 - set2): {diferencia_sets}") # Salida: Diferencia (set1 - set2): {1, 2} print(f"Diferencia con -: {set1 - set2}") # Salida: Diferencia con -: {1, 2}

symmetric_difference(otro_conjunto) o ^: Devuelve un nuevo conjunto con los elementos que están en uno de los conjuntos, pero no en ambos. python diferencia_simetrica = set1.symmetric_difference(set2) print(f"Diferencia simétrica: {diferencia_simetrica}") # Salida: Diferencia simétrica: {1, 2, 4, 5} print(f"Diferencia simétrica con

`^: {set1 ^ set2}") # Salida: Diferencia simétrica con ^: {1, 2, 4, 5}`

issubset(otro_conjunto): Comprueba si todos los elementos del conjunto actual están en el otro conjunto.

- **issuperset(otro_conjunto):** Comprueba si todos los elementos del otro conjunto están en el conjunto actual.

4.5. ¿Cuándo usar Conjuntos?

Los conjuntos son ideales para:

- **Eliminar duplicados** de una colección.
- Realizar **pruebas de pertenencia** rápidas (es muy eficiente verificar si un elemento está en un conjunto).
- Realizar **operaciones matemáticas de conjuntos** (unión, intersección, diferencia).

5. Comparación y Resumen

Aquí tienes una tabla que resume las características clave de Listas, Tuplas y Conjuntos:

| Característica | Listas ([]) | Tuplas (()) | Conjuntos ({ } o set()) |
|--------------------|---|--|---|
| Orden | Sí (se mantiene el orden de inserción) | Sí (se mantiene el orden de inserción) | No (los elementos no tienen un orden fijo) |
| Mutabilidad | Mutable (se pueden añadir, eliminar, modificar elementos) | Immutable (no se pueden cambiar los elementos una vez creada) | Mutable (se pueden añadir/eliminar elementos del conjunto) |
| Permite Duplicados | Sí | Sí | No (solo almacena elementos únicos) |
| Sintaxis | Corchetes [] | Paréntesis () | Llaves { } (o set() para vacío) |
| Uso Típico | Colecciones de elementos que cambian, pilas, colas, datos heterogéneos. | Colecciones de elementos fijos, registros, claves de diccionario (si son inmutables), valores de retorno de funciones. | Almacenar elementos únicos, pruebas de pertenencia rápidas, operaciones de conjuntos (unión, intersección). |
| Acceso por Índice | Sí | Sí | No |

Conclusión

Hemos explorado tres tipos de colecciones fundamentales en Python: Listas, Tuplas y Conjuntos. Comprender sus diferencias en cuanto a orden, mutabilidad y manejo de duplicados es crucial para elegir la herramienta adecuada para cada tarea de programación.

- Usa **Listas** cuando necesites una colección ordenada y flexible que pueda cambiar.
- Usa **Tuplas** cuando necesites una colección ordenada de elementos que no deben cambiar.
- Usa **Conjuntos** cuando necesites una colección de elementos únicos y el orden no sea importante.

En la próxima lección, exploraremos otra colección poderosa: los **Diccionarios**, que nos permiten almacenar datos en pares clave-valor.

¡Sigán practicando con estos conceptos! La manipulación de colecciones es una habilidad central en cualquier lenguaje de programación.

2.2 Lección 3.2: Diccionarios

¡Excelente! Bienvenidos a nuestra Lección 3.2, donde profundizaremos en una de las estructuras de datos más potentes y versátiles de Python: los **Diccionarios**. Prepárense para descubrir cómo organizar y acceder a sus datos de una manera increíblemente eficiente.

Lección 3.2: Diccionarios en Python - Almacenando Datos con Clave-Valor

Introducción: ¿Qué son los Diccionarios?

Imagina que tienes una agenda telefónica. Para encontrar el número de teléfono de alguien, no buscas por la página 1, luego la 2, etc. En su lugar, buscas directamente por el **nombre** de la persona. Ese nombre es tu "clave", y el número de teléfono es el "valor" asociado a esa clave.

En Python, un **diccionario** funciona de manera muy similar. Es una colección de elementos donde cada elemento es un **par clave-valor**. A diferencia de las listas o las tuplas, donde los elementos se acceden por su posición numérica (índice), en los diccionarios se accede a los valores directamente a través de sus **claves**. Esto los hace ideales para almacenar datos que tienen una relación lógica entre una etiqueta (la clave) y la información asociada (el valor).

Creación de Diccionarios

Los diccionarios se definen utilizando llaves `{ }` y cada par clave-valor se separa por dos puntos `:`, mientras que los pares entre sí se separan por comas `,`.

Sintaxis básica:

```
python mi_diccionario = {clave1: valor1, clave2: valor2, clave3: valor3}
```

Ejemplo:

```
```python
```

### Un diccionario simple con información de una persona

---

```
persona = { "nombre": "Alice", "edad": 30, "ciudad": "Nueva York" } print(persona)
```

Salida: {'nombre': 'Alice', 'edad': 30, 'ciudad': 'Nueva York'}

---

### Un diccionario de películas y sus directores, como el del contexto

---

```
peliculas = { "Love Actually": "Richard Curtis", "Amélie": "Jean-Pierre Jeunet" } print(peliculas)
```

Salida: {'Love Actually': 'Richard Curtis', 'Amélie': 'Jean-Pierre Jeunet'}

---

### Un diccionario vacío

---

```
diccionariovacio = {} print(diccionariovacio)
```

Salida: {}

---

```
```
```

Características Clave de los Diccionarios

Pares Clave-Valor: Cada entrada en un diccionario es un par donde la primera parte es la **clave** y la segunda es el **valor** asociado a esa clave.

- **Clave:** Debe ser única dentro del diccionario. Si intentas añadir una clave que ya existe, el valor anterior será sobrescrito.
- **Valor:** Puede ser cualquier tipo de dato en Python (números, cadenas, listas, otros diccionarios, etc.).

Acceso por Clave, No por Índice: Esta es la diferencia fundamental con listas y tuplas. Para obtener un valor, se utiliza la clave entre corchetes `[]`.

```
```python
```

### Usando el diccionario 'peliculas'

---

```
directorloveactually = peliculas["Love Actually"] print(f"El director de 'Love Actually' es: {directorloveactually}")
```

Salida: El director de 'Love Actually' es: Richard Curtis

---

Si intentamos acceder a una clave que no existe, obtendremos un error `KeyError`

---

```
director_titanic = peliculas["Titanic"] # Esto causaría un error
```

---

```
```
```

Claves Inmutables: ¡Este es un punto crucial! Las claves de un diccionario deben ser **objetos inmutables**. Esto significa que no pueden cambiar después de ser creadas.

- **Tipos de datos que pueden ser claves:** Números (enteros, flotantes), cadenas de texto (strings), booleanos, y tuplas (siempre que todos los elementos de la tupla sean también inmutables).
- **Tipos de datos que NO pueden ser claves:** Listas, diccionarios, y cualquier otro objeto mutable.

¿Por qué esta restricción? Los diccionarios en Python se implementan utilizando una técnica llamada **tablas hash**. Cuando se introduce un nuevo par clave-valor, se calcula un "hash" (una especie de huella digital numérica) de la clave. Este hash se utiliza para encontrar rápidamente la ubicación donde se almacenará el valor. Si la clave fuera mutable y cambiara después de haber sido introducida, su hash también cambiaría, y el diccionario ya no podría encontrar el valor asociado, ya que buscaría en una ubicación incorrecta.

```
```python
```

## Ejemplos de claves válidas

---

```
diccionariovariado = { 1: "Número", "nombre": "Cadena", True: "Booleano", (1, 2): "Tupla inmutable" } print(diccionariovariado)
```

## Ejemplo de clave inválida (lista)

---

```
diccionario_invalido = {[1, 2]: "Lista como clave"} # Esto causaría un error TypeError
```

---

```
```
```

No Ordenados (Históricamente, pero con Matices en Python Moderno): Históricamente, los diccionarios no garantizaban un orden de inserción. Sin embargo, a partir de **Python 3.7**, los diccionarios sí mantienen el orden en que se insertaron los elementos. Esto significa que si iteras sobre un diccionario, los elementos aparecerán en el mismo orden en que los añadiste.

Operaciones Básicas con Diccionarios

1. Acceder a Valores

Ya vimos el acceso directo con `[]`. Otra forma más segura es usar el método `get()`.

- `d[c_lave]`: Devuelve el valor asociado a `c_lave`. Si la clave no existe, lanza un `KeyError`.
- `d.get(c_lave, valor_por_defecto)`: Devuelve el valor asociado a `c_lave`. Si la clave no existe, devuelve `None` o el `valor_por_defecto` especificado.

```
```python print(peliculas["Amélie"]) # Acceso directo
```

## Salida: Jean-Pierre Jeunet

---

```
print(peliculas.get("Titanic")) # Clave no existe, devuelve None
```

## Salida: None

---

```
print(peliculas.get("Titanic", "Director Desconocido")) # Clave no existe, devuelve valor por defecto
```

## Salida: Director Desconocido

---

```
```
```

2. Añadir y Modificar Elementos

Para añadir o modificar un elemento, simplemente asigna un valor a una clave.

- Si la `c_lave` no existe, se añade un nuevo par clave-valor.

- Si la clave ya existe, su valor asociado se actualiza.

```
```python
```

### Añadir una nueva película

---

```
películas["El Padrino"] = "Francis Ford Coppola" print(películas)
```

**Salida: {'Love Actually': 'Richard Curtis', 'Amélie': 'Jean-Pierre Jeunet', 'El Padrino': 'Francis Ford Coppola'}**

---

### Modificar el director de una película existente

---

```
películas["Love Actually"] = "Richard Curtis (Edición Especial)" print(películas)
```

**Salida: {'Love Actually': 'Richard Curtis (Edición Especial)', 'Amélie': 'Jean-Pierre Jeunet', 'El Padrino': 'Francis Ford Coppola'}**

---

```
```
```

3. Eliminar Elementos

- `del d[clave]`: Elimina el par clave-valor. Si la clave no existe, lanza un `KeyError`.
- `d.pop(clave, valor_por_defecto)`: Elimina el par clave-valor y devuelve el valor asociado. Si la clave no existe, lanza un `KeyError` a menos que se proporcione un `valor_por_defecto`.
- `d.clear()`: Elimina todos los elementos del diccionario, dejándolo vacío.

```
```python
```

### Eliminar una película usando del

---

```
del películas["El Padrino"] print(películas)
```

**Salida: {'Love Actually': 'Richard Curtis (Edición Especial)', 'Amélie': 'Jean-Pierre Jeunet'}**

---

### Eliminar una película usando pop y obtener su valor

---

```
directoramelie = películas.pop("Amélie") print(f"El director de Amélie era: {directoramelie}") print(películas)
```

**Salida: El director de Amélie era: Jean-Pierre Jeunet**

---

**Salida: {'Love Actually': 'Richard Curtis (Edición Especial)'}**

---

### Intentar eliminar una clave que no existe con pop (sin valor por defecto)

---

```
películas.pop("Titanic") # Esto causaría un KeyError
```

---

**Eliminar una clave que no existe con pop (con valor por defecto)**

---

```
directortitanic = peliculas.pop("Titanic", "No encontrada") print(f"Intento de eliminar Titanic: {directortitanic}")
```

**Salida: Intento de eliminar Titanic: No encontrada**

---

**Vaciar el diccionario**

---

```
peliculas.clear() print(peliculas)
```

**Salida: {}**

---

...

#### 4. Verificar la Existencia de una Clave (k in D)

Esta es una operación muy común y eficiente para comprobar si una clave está presente en el diccionario.

- `clave in diccionario`: Devuelve True si la clave existe, False en caso contrario.

```
``python frutas = {"manzana": 10, "banana": 5, "naranja": 8}

print("manzana" in frutas)
```

**Salida: True**

---

```
print("uva" in frutas)
```

**Salida: False**

---

```
if "banana" in frutas: print(f"Tenemos {frutas['banana']} bananas.")
```

**Salida: Tenemos 5 bananas.**

---

...

#### 5. Obtener Todos los Pares Clave-Valor (D.items())

El método `items()` devuelve una vista de los elementos del diccionario como una lista de tuplas, donde cada tupla contiene un par (`clave`, `valor`). Es muy útil para iterar sobre el diccionario.

```
``python frutas = {"manzana": 10, "banana": 5, "naranja": 8}
```

**Imprimir la vista de items**

---

```
print(frutas.items())
```

**Salida: dict\_items([('manzana', 10), ('banana', 5), ('naranja', 8)])**

---

**Iterar sobre los items**

---

```
print("\nInventario de frutas:") for clave, valor in frutas.items(): print(f"- {clave.capitalize(): {valor} unidades")
```

**Salida:**

---

- Manzana: 10 unidades

---

- Banana: 5 unidades

---

- Naranja: 8 unidades

---

...

Además de `items()`, también existen `d.keys()` (devuelve una vista de todas las claves) y `d.values()` (devuelve una vista de todos los valores).

```
```python print(frutas.keys())
```

Salida: dict_keys(['manzana', 'banana', 'naranja'])

```
print(frutas.values())
```

Salida: dict_values([10, 5, 8])

...

Ejemplo Práctico Integrado: Gestión de un Inventario Simple

Vamos a aplicar lo aprendido para gestionar un pequeño inventario de productos.

```
```python
```

#### 1. Crear el inventario inicial

---

```
inventario = { "laptop": 15, "raton": 50, "teclado": 25, "monitor": 10 } print("--- Inventario Inicial ---") for producto, cantidad in inventario.items(): print(f"{producto.capitalize(): {cantidad} unidades")
```

#### 2. Añadir un nuevo producto

---

```
inventario["webcam"] = 30 print("\n--- Después de añadir Webcam ---") print(inventario)
```

#### 3. Actualizar la cantidad de un producto existente

---

```
inventario["laptop"] += 5 # Se compraron 5 laptops más print("\n--- Después de actualizar Laptops ---") print(inventario)
```

#### 4. Verificar si un producto está en el inventario

---

```
productoabuscar = "raton" if productoabuscar in inventario: print(f"\nSí, tenemos {inventario[productoabuscar]} unidades de {productoabuscar}.") else: print(f"\nEl producto {productoabuscar} no está en el inventario.")
```

```
productoabuscar = "impresora" if productoabuscar in inventario: print(f"\nSí, tenemos {inventario[productoabuscar]} unidades de {productoabuscar}.") else: print(f"\nEl producto {productoabuscar} no está en el inventario.")
```

#### 5. Eliminar un producto que se agotó

---

```
productoagotado = "teclado" if productoagotado in inventario: cantidadeliminada = inventario.pop(productoagotado) print(f"\nSe eliminó {productoagotado} del inventario. Había {cantidadeliminada} unidades.") else: print(f"\n{producto_agotado} no estaba en el inventario para
```

eliminar:")

```
print("\n--- Inventario Final ---") for producto, cantidad in inventario.items(): print(f"{producto.capitalize(): {cantidad} unidades") ``
```

## Resumen y Conclusión

Los diccionarios son una herramienta indispensable en Python para organizar datos de forma lógica y eficiente.

- Permiten almacenar información en **pares clave-valor**.
- El acceso a los datos se realiza mediante la **clave**, no por un índice numérico.
- Las **claves deben ser inmutables** (cadenas, números, tuplas, etc.) debido a su implementación con tablas hash.
- Ofrecen métodos sencillos para **añadir, modificar, eliminar y verificar** la existencia de elementos.
- `D.items()` es muy útil para **iterar** sobre todos los pares clave-valor.

Dominar los diccionarios abrirá un mundo de posibilidades para estructurar y manipular datos complejos en sus programas. ¡Sigán practicando!

---

## Ejercicio Propuesto

Crea un diccionario llamado `estudiante` que contenga la siguiente información: `* "nombre": "Ana García" * "edad": 22 * "carrera": "Ingeniería Informática" * "cursos": ["Programación I", "Bases de Datos", "Redes"]`

Luego, realiza las siguientes operaciones: 1. Imprime el diccionario completo. 2. Accede e imprime la carrera del estudiante. 3. Añade una nueva clave `"promedio"` con el valor `9.2`. 4. Modifica la edad a `23`. 5. Verifica si la clave `"universidad"` existe en el diccionario. Si no existe, añádela con el valor `"Universidad Nacional"`. 6. Imprime el diccionario final, iterando sobre sus ítems para mostrar cada clave y su valor.

## 2.3 Lección 3.3: Manejo de Archivos

¡Excelente! Bienvenidos a nuestra clase de hoy. Como su profesor experto, los guiaré a través de los fundamentos y técnicas avanzadas para interactuar con archivos en sus programas.

---

## Lección 3.3: Manejo de Archivos - Lectura, Escritura y Posicionamiento

---

### Introducción al Manejo de Archivos

En el desarrollo de software, la capacidad de interactuar con el sistema de archivos es fundamental. Nos permite almacenar datos de forma persistente, más allá de la ejecución de un programa, y recuperar información previamente guardada. Hoy nos centraremos en los archivos de texto, que son los más comunes para almacenar configuraciones, logs o datos estructurados de forma sencilla.

Al finalizar esta lección, serán capaces de:   
\* Abrir y cerrar archivos de manera segura.   
\* Escribir contenido en archivos de texto, incluyendo listas de líneas.   
\* Leer contenido de archivos de texto de diversas formas.   
\* Mover el puntero de lectura/escritura dentro de un archivo para operaciones avanzadas.   
\* Utilizar herramientas de inspección para entender mejor los objetos de Python.

---

## 1. El Objeto `file` y el Ciclo de Vida de un Archivo

Cuando trabajamos con archivos, Python nos proporciona un objeto especial, el **objeto `file`** (o `TextIOWrapper` en versiones modernas), que actúa como una interfaz entre nuestro programa y el archivo físico en el disco.

### 1.1. Abriendo un Archivo (`open()`)

Para comenzar a interactuar con un archivo, primero debemos "abrirlo" utilizando la función `open()`. Esta función toma al menos dos parámetros: la ruta del archivo y el modo de apertura.

```
```python
```

Sintaxis básica

```
archivo = open('nombredearchivo.txt', 'modo') ``
```

Modos de Apertura Comunes:

- 'r' (read): Abre el archivo para lectura. Si el archivo no existe, se produce un error (`FileNotFoundError`). El puntero se coloca al principio.
- 'w' (write): Abre el archivo para escritura. **Si el archivo ya existe, su contenido se truncará (se borrará)**. Si no existe, se crea uno nuevo. El puntero se coloca al principio.
- 'a' (append): Abre el archivo para añadir contenido. Si el archivo no existe, se crea uno nuevo. El puntero se coloca al final del archivo.
- 'x' (exclusive creation): Crea un nuevo archivo para escritura. Si el archivo ya existe, se produce un error (`FileExistsError`).
- 'r+' (read and write): Abre el archivo para lectura y escritura. El puntero se coloca al principio.
- 'w+' (write and read): Abre el archivo para escritura y lectura. **Si el archivo ya existe, su contenido se truncará**. Si no existe, se crea uno nuevo. El puntero se coloca al principio.
- 'a+' (append and read): Abre el archivo para añadir y leer. Si el archivo no existe, se crea uno nuevo. El puntero se coloca al final del archivo.

Codificación (Encoding): Es una buena práctica especificar la codificación del archivo, especialmente para asegurar la compatibilidad con caracteres especiales. `utf-8` es la codificación estándar y recomendada.

```
python archivo = open('mi_documento.txt', 'w', encoding='utf-8')
```

1.2. Cerrando un Archivo (`.close()`)

Después de realizar las operaciones necesarias, es **crucial cerrar el archivo** para liberar los recursos del sistema y asegurar que todos los cambios se guarden correctamente.

```
python archivo = open('mi_documento.txt', 'w', encoding='utf-8') archivo.write("Hola, este es mi primer archivo.") archivo.close() # ¡Importante!
```

1.3. El Gestor de Contexto (`with statement`) - La Mejor Práctica

Olvidar cerrar un archivo puede llevar a problemas como corrupción de datos o fugas de recursos. Python ofrece una solución elegante y segura: el gestor de contexto `with`.

Cuando se usa `with`, el archivo se cierra automáticamente al salir del bloque `with`, incluso si ocurre un error.

```
``python with open('mi_documento.txt', 'w', encoding='utf-8') as archivo: archivo.write("Este texto se escribe de forma segura.\n")
archivo.write("El archivo se cerrará automáticamente.")
```

```
print("Archivo cerrado automáticamente.") ``
```

De ahora en adelante, utilizaremos siempre el `with statement` para manejar archivos.

2. Escritura en Archivos de Texto

Una vez que tenemos un archivo abierto en modo de escritura ('w', 'a', 'w+', 'a+'), podemos añadir contenido a él.

2.1. Escribir una Sola Cadena (`.write()`)

El método `write()` toma una cadena de texto como parámetro y la escribe en el archivo en la posición actual del puntero. Es importante recordar que `write()` no añade automáticamente un salto de línea; debemos incluir `\n` explícitamente si queremos que el texto aparezca en líneas separadas.

```
``python with open('saludo.txt', 'w', encoding='utf-8') as f: f.write("¡Hola, mundo!\n") f.write("Esta es la segunda línea.\n") f.write("Y esta es la tercera.")
```

```
print("Contenido escrito en 'saludo.txt'.") ``
```

2.2. Escribir Múltiples Líneas (`.writelines()`)

El método `writelines()` es muy útil cuando queremos escribir una **lista de cadenas de texto** en el archivo. Cada cadena en la lista se escribe secuencialmente. Al igual que con `write()`, `writelines()` no añade saltos de línea automáticamente, por lo que cada cadena en la lista debe contener `\n` si deseamos que se escriban en líneas separadas.

Contexto Técnico Relevante: El método `writelines()` toma como parámetro una lista de cadenas de texto, indicando las líneas que queremos escribir en el fichero.

```
python lineasaescribir = [ "Primera línea de mi lista.\n", "Segunda línea con información.\n", "Tercera y última línea de este bloque.\n" ]

with open('listadecompras.txt', 'w', encoding='utf-8') as f: f.writelines(lineasaescribir) f.write("--- Fin de la lista ---\n")

print("Lista de líneas escrita en 'listadecompras.txt'.")
```

Ejemplo de añadir a un archivo existente

```
nuevas_lineas = [ "Manzanas\n", "Leche\n", "Pan\n" ]

with open('listadecompras.txt', 'a', encoding='utf-8') as f: # Modo 'a' para añadir f.write("\n--- Productos adicionales ---\n")
f.writelines(nuevas_lineas)

print("Productos adicionales añadidos a 'listadecompras.txt'.")
```

3. Lectura de Archivos de Texto

Para leer contenido de un archivo, debemos abrirlo en modo de lectura ('r', 'r+', 'w+', 'a+').

3.1. Leer el Contenido Completo (.read())

El método `read()` lee todo el contenido del archivo y lo devuelve como una única cadena de texto.

```
python with open('saludo.txt', 'r', encoding='utf-8') as f: contenido_completo = f.read() print("---
Contenido de saludo.txt ---") print(contenido_completo)
```

3.2. Leer una Sola Línea (.readline())

El método `readline()` lee una línea del archivo cada vez que se llama, incluyendo el carácter de salto de línea (`\n`) al final. Cuando no quedan más líneas, devuelve una cadena vacía ('').

```
python with open('lista_de_compras.txt', 'r', encoding='utf-8') as f: print("\n--- Leyendo línea por línea
con readline() ---") linea1 = f.readline() linea2 = f.readline() linea3 = f.readline() print(f"Línea 1:
{linea1.strip()}") # .strip() para quitar \n print(f"Línea 2: {linea2.strip()}") print(f"Línea 3:
{linea3.strip()}")
```

3.3. Leer Todas las Líneas en una Lista (.readlines())

El método `readlines()` lee todas las líneas del archivo y las devuelve como una lista de cadenas, donde cada cadena representa una línea del archivo (incluyendo el `\n`).

```
python with open('lista_de_compras.txt', 'r', encoding='utf-8') as f: todas_las_lineas = f.readlines()
print("\n--- Leyendo todas las líneas con readlines() ---") for i, linea in enumerate(todas_las_lineas):
print(f"Línea {i+1}: {linea.strip()}")
```

3.4. Iterar Directamente sobre el Objeto Archivo (Más Pythonico)

La forma más común y eficiente de leer un archivo línea por línea es iterar directamente sobre el objeto archivo. Esto lee el archivo de forma perezosa (lazy loading), lo que significa que no carga todo el archivo en memoria a la vez, siendo ideal para archivos grandes.

```
python with open('lista_de_compras.txt', 'r', encoding='utf-8') as f: print("\n--- Iterando sobre el
objeto archivo ---") for i, linea in enumerate(f): print(f"Línea {i+1} (iteración): {linea.strip()}")
```

4. Moviendo el Puntero de Lectura/Escritura (.seek() y .tell())

Imagina que el archivo es una cinta de casete y el puntero es el cabezal de lectura/escritura. Normalmente, este puntero avanza a medida que leemos o escribimos. Sin embargo, hay situaciones en las que nos puede interesar mover el puntero de lectura/escritura a una posición determinada del archivo. Por ejemplo, si queremos empezar a escribir en una posición determinada y no al final o al principio del archivo, o si

queremos releer una sección.

4.1. Conociendo la Posición Actual (.tell())

El método `.tell()` nos devuelve la posición actual del puntero dentro del archivo, medida en bytes desde el principio del archivo.

```
```python with open('ejemplo_puntero.txt', 'w+', encoding='utf-8') as f: f.write("ABCDEFGH") print(f"Posición actual después de escribir 'ABCDEFGH': {f.tell()}") # Debería ser 8

f.seek(0) # Mover al principio
print(f"Posición actual después de seek(0): {f.tell()}") # Debería ser 0

contenido = f.read(3) # Leer 3 caracteres
print(f"Contenido leído: '{contenido}'")
print(f"Posición actual después de leer 3 caracteres: {f.tell()}") # Debería ser 3

...`
```

#### 4.2. Moviendo el Puntero (.seek())

Para esto se utiliza el método `seek()`. Este método toma como parámetro un número entero que representa el desplazamiento (offset) en bytes. Opcionalmente, puede tomar un segundo parámetro whence que indica desde dónde se calcula el desplazamiento:

- 0 (por defecto): Desde el principio del archivo.
- 1: Desde la posición actual del puntero.
- 2: Desde el final del archivo.

**Contexto Técnico Relevante:** El método `seek` toma como parámetro un número (el desplazamiento en bytes).

```
```python with open('ejemplo_seek.txt', 'w+', encoding='utf-8') as f: f.write("Python es genial para el manejo de archivos.") print(f"Posición inicial: {f.tell()}") # 44 (longitud de la cadena)

# Mover el puntero al principio (posición 0)
f.seek(0)
print(f"Posición después de seek(0): {f.tell()}") # 0

# Leer los primeros 6 caracteres
primeros_seis = f.read(6)
print(f"Leído: '{primeros_seis}'") # "Python"
print(f"Posición actual: {f.tell()}") # 6

# Mover el puntero a la posición 10 (desde el principio)
f.seek(10)
print(f"Posición después de seek(10): {f.tell()}") # 10

# Leer 2 caracteres desde la posición 10
dos_chars = f.read(2)
print(f"Leído: '{dos_chars}'") # "es"
print(f"Posición actual: {f.tell()}") # 12

# Ejemplo de escritura en una posición específica
f.seek(7) # Mover a la posición 7 (después de 'Python ')
f.write("es MUY genial") # Sobreescribe "es genial"
f.seek(0) # Volver al principio para leer todo
contenido_final = f.read()
print(f"\nContenido final del archivo después de sobreescritura:\n'{contenido_final}'")
# Resultado esperado: "Python es MUY genial para el manejo de archivos."
# Noten que "para el manejo de archivos." se mantiene porque la nueva escritura fue más corta.

...`
```

Consideraciones con seek(): * Cuando se trabaja con archivos de texto, `seek()` y `tell()` pueden ser un poco más complejos debido a la codificación de caracteres (un carácter puede no ser siempre un byte). Para la mayoría de los casos con `utf-8` y caracteres ASCII, funcionará como se espera. Para una precisión absoluta con caracteres multi-byte, a veces es mejor trabajar con archivos binarios o tener cuidado con los offsets. * El modo `w` (escritura) trunca el archivo al abrirlo, por lo que `seek()` solo tiene sentido después de haber escrito algo o si se usa un

modo como `r+`, `w+`, `a+`.

5. El Operador `?` para Inspección de Objetos

Como programadores, a menudo necesitamos recordar cómo funciona un objeto, una función o un método. Python, especialmente en entornos interactivos como IPython o Jupyter Notebooks, nos ofrece una herramienta muy útil para esto: el operador `?`.

El operador `?` nos muestra información sobre los objetos. Se utiliza añadiendo el símbolo de interrogación al final del nombre del objeto, función o método del cual queremos más información.

Contexto Técnico Relevado: El operador `?` nos muestra información sobre los objetos. Se utiliza añadiendo el símbolo de interrogación al final del nombre del objeto del cual queremos más información. Por ejemplo: `In [3]: str?`

Ejemplos de Uso:

```
```python
```

#### Para obtener información sobre el tipo de dato `string`

---

```
str?
```

#### Para obtener información sobre el método `.write()` de un objeto `file`

---

##### Primero, necesitamos un objeto `file`

---

```
with open('temp.txt', 'w') as f: f.write? # Ejecutar esto en un entorno interactivo como IPython/Jupyter
```

#### Para obtener información sobre la función `open()`

---

```
open?
```

#### Para obtener información sobre una variable

---

```
milista = [1, 2, 3] milista?
```

#### Para obtener información sobre el método `.seek()`

---

```
with open('temp.txt', 'r+') as f: f.seek? # Ejecutar esto en un entorno interactivo ```
```

Al ejecutar `objeto?` en un entorno interactivo, se mostrará la docstring (cadena de documentación) del objeto, su tipo, la firma de la función (si es una función o método) y, a veces, incluso el archivo fuente donde está definido. Es una herramienta invaluable para la exploración y el aprendizaje.

---

## Conclusión

---

Hemos cubierto los aspectos fundamentales del manejo de archivos de texto en Python. Desde la apertura y cierre seguros con el `with` statement, pasando por la escritura de contenido (incluyendo listas de líneas con `writelines()`) y la lectura en diversas modalidades, hasta el control preciso del puntero con `seek()` y `tell()`. Además, hemos aprendido a usar el operador `?` para obtener ayuda instantánea sobre cualquier objeto en Python.

Dominar estas técnicas es esencial para cualquier desarrollador, ya que la persistencia de datos es un requisito casi universal en las aplicaciones.

---

## Ejercicios Propuestos

---

### Crear y Leer un Archivo de Configuración:

- Crea un archivo llamado `config.txt` utilizando el modo de escritura ('w').
- Escribe las siguientes líneas en él usando `writelines()`: `DATABASE_HOST=localhost DATABASE_PORT=5432`  
`USERNAME=admin PASSWORD=secure_password`
- Cierra el archivo.
- Abre el archivo `config.txt` en modo de lectura ('r').
- Lee el contenido línea por línea utilizando un bucle `for` y muestra cada línea en la consola, eliminando los saltos de línea.

### Registro de Eventos con `seek()` y `tell()`:

- Crea un archivo llamado `log.txt` en modo `w+` (lectura y escritura, truncando si existe).
- Escribe la cadena "Inicio del programa\n" en el archivo.
- Usa `tell()` para obtener la posición actual del puntero y guárdala en una variable.
- Escribe la cadena "Procesando datos...\n" en el archivo.
- Usa `seek()` para volver a la posición guardada anteriormente.
- Escribe la cadena "Error crítico detectado!\n" en esa posición, sobrescribiendo parte del texto original.
- Vuelve al principio del archivo (`seek(0)`) y lee todo el contenido.
- Imprime el contenido final del archivo. ¿Qué observas?

### Exploración con ?:

- En un entorno interactivo (como IPython o Jupyter), crea una variable `mi_cadena = "Hola Python"`.
- Usa `mi_cadena?` para ver su información.
- Usa `mi_cadena.upper?` para ver la información del método `upper()`.
- Crea un objeto `f` abriendo un archivo temporal (`with open('temp_file.txt', 'w') as f:`). Luego, fuera del bloque `with`, intenta `f.write?` (esto fallará porque el archivo ya está cerrado, lo cual es una buena lección). Luego, abre un archivo *sin* `with` y prueba `f.write?` y `f.seek?`.

¡Mucha suerte con los ejercicios! La práctica es clave para dominar estas herramientas.

## Módulo 3: Módulo 4: Programación Orientada a Objetos y Manejo de Errores

---

### 3.1 Lección 4.1: Clases y Objetos

¡Excelente! Es un placer guiarte en este fascinante viaje hacia la Programación Orientada a Objetos. Prepárate para cambiar tu forma de pensar sobre cómo construir software.

---

## Lección 4.1: Clases y Objetos - Los Pilares de la Programación Orientada a Objetos (POO)

---

¡Bienvenidos a la Lección 4.1! Hasta ahora, hemos trabajado principalmente con la programación estructurada, un paradigma que nos ha permitido resolver problemas de manera secuencial y modular mediante funciones. Sin embargo, Python, como lenguaje multiparadigma, nos ofrece herramientas mucho más potentes para modelar el mundo real: la **Programación Orientada a Objetos (POO)**.

En esta lección, desglosaremos los conceptos fundamentales de la POO: qué son las **clases** y los **objetos**, cómo se relacionan y cómo podemos definirlos y utilizarlos en Python.

---

### 1. ¿Qué es la Programación Orientada a Objetos (POO)?

---

La Programación Orientada a Objetos (POO, u OOP por sus siglas en inglés) es un **paradigma de programación** que busca modelar los conceptos del mundo real relevantes para nuestro problema a través de entidades llamadas **clases** y **objetos**. En un programa POO, la solución se construye como una serie de interacciones entre estos objetos.

**¿Por qué POO?** La POO ofrece ventajas significativas: \* **Modularidad**: Permite dividir un programa complejo en módulos más pequeños y manejables (objetos). \* **Reusabilidad**: Una vez que defines una clase, puedes crear múltiples objetos a partir de ella, y las clases pueden ser reutilizadas en diferentes partes del programa o incluso en otros proyectos. \* **Mantenibilidad**: Los cambios en una parte del sistema tienen un

impacto limitado en otras partes, facilitando la depuración y actualización. \* **Abstracción:** Nos permite enfocarnos en lo que un objeto *hace* en lugar de cómo *lo hace*, ocultando los detalles internos complejos.

---

## 2. Clases y Objetos: Entendiendo la Diferencia

---

Para adentrarnos en la POO, es crucial comprender la distinción entre una clase y un objeto.

### 2.1. ¿Qué es un Objeto?

Un **objeto** es una entidad concreta que agrupa un **estado** y una **funcionalidad** relacionadas. Piensa en cualquier cosa a tu alrededor: tu teléfono, una silla, un perro, un coche. Cada uno de ellos es un objeto.

**Estado (Atributos):** El estado de un objeto se define a través de variables internas, llamadas **atributos**. Estos describen las características o propiedades del objeto.

- *Ejemplo:* Para un objeto "coche", sus atributos podrían ser `marca`, `modelo`, `color`, `velocidad_actual`, `nivel_gasolina`.

**Funcionalidad (Métodos):** La funcionalidad de un objeto se modela a través de funciones, a las que se les conoce como **métodos**. Estos describen las acciones que el objeto puede realizar o las acciones que se pueden realizar sobre él.

- *Ejemplo:* Para un objeto "coche", sus métodos podrían ser `arrancar()`, `acelerar()`, `frenar()`, `repostar()`.

### 2.2. ¿Qué es una Clase?

Si un objeto es una entidad concreta, una **clase** es la **plantilla**, el **molde** o el **plano** a partir del cual se crean (o "instancian") los objetos. La clase define qué atributos y métodos tendrán todos los objetos de ese tipo.

- Una clase no es un objeto en sí misma, sino la **definición** de cómo deben ser los objetos de una categoría específica.
- Piensa en el plano de una casa (la clase). Puedes construir muchas casas idénticas o ligeramente diferentes (los objetos) a partir de ese mismo plano. Cada casa es una instancia única del plano.

### 2.3. Ejemplo del Mundo Real: El Coche

Retomemos el ejemplo del coche que se mencionaba en el contexto:

#### Clase: Coche

- Representa la idea abstracta de "coche".
- Define que *todos* los coches tendrán atributos como `marca`, `modelo`, `color`, `velocidad_actual`, `nivel_gasolina`.
- Define que *todos* los coches podrán realizar acciones como `arrancar()`, `acelerar()`, `frenar()`, `repostar()`.

#### Objetos (Instancias):

`mi_coche`: Un objeto concreto de la clase Coche.

- Atributos: `marca="Toyota"`, `modelo="Corolla"`, `color="Rojo"`, `velocidad_actual=0`, `nivel_gasolina=50`.

`coche_vecino`: Otro objeto concreto de la clase Coche.

- Atributos: `marca="Ford"`, `modelo="Fiesta"`, `color="Azul"`, `velocidad_actual=0`, `nivel_gasolina=30`.

Aunque `mi_coche` y `coche_vecino` pertenecen a la misma clase (Coche), son objetos distintos con sus propios estados (sus propios valores para `marca`, `modelo`, etc.).

---

## 3. Definiendo Clases en Python

---

En Python, las clases se definen utilizando la palabra clave `class`, seguida del nombre de la clase y dos puntos (`:`). El cuerpo de la clase debe estar indentado.

### 3.1. Sintaxis Básica

```
python class NombreDeLaClase: """Cadena de documentación (docstring) de la clase. Describe el propósito de la clase. """ # Cuerpo de la clase: aquí se definen atributos y métodos pass # 'pass' es una instrucción nula, se usa cuando el cuerpo está vacío temporalmente
```

- **Convención de Nombres:** Por convención, los nombres de las clases en Python se escriben en **PascalCase** (la primera letra de cada palabra en mayúscula, sin espacios), por ejemplo: `MiClase`, `CocheDeportivo`.
- **Docstring:** Al igual que con las funciones, la primera línea del cuerpo de la clase puede ser una cadena de texto (docstring) que describe la clase. Es una buena práctica documentar tus clases.

### 3.2. El Método `__init__` (Constructor)

El método `__init__` es un método especial en Python, conocido como el **constructor**. Se llama automáticamente cada vez que se crea un nuevo objeto (una nueva instancia) de la clase. Su propósito principal es **inicializar el estado del objeto**, es decir, asignar valores iniciales a sus atributos.

```
```python
class Coche: """Abstracción de los objetos coche."""

    def __init__(self, marca, modelo, color, gasolina_inicial):
        """
        Método constructor para inicializar un objeto Coche.

        Args:
            marca (str): La marca del coche.
            modelo (str): El modelo del coche.
            color (str): El color del coche.
            gasolina_inicial (int): La cantidad inicial de gasolina en litros.
        """
        # Atributos de instancia
        self.marca = marca
        self.modelo = modelo
        self.color = color
        self.gasolina = gasolina_inicial
        self.velocidad_actual = 0 # Atributo con valor inicial por defecto

        print(f"¡Un nuevo {self.color} {self.marca} {self.modelo} ha sido creado!")
        print(f" Tiene {self.gasolina} litros de gasolina y está parado.")
    ...
```

- **self:** Este es el primer parámetro de cualquier método dentro de una clase (incluido `__init__`). `self` es una convención (podría llamarse de otra forma, pero no es recomendable) y representa la **propia instancia del objeto** que se está creando o sobre el que se está operando. A través de `self`, podemos acceder y modificar los atributos y llamar a otros métodos del objeto.
- **Atributos de Instancia:** Los atributos que pertenecen a un objeto específico se crean prefijándolos con `self`. (ej. `self.marca = marca`). Esto los distingue de las variables locales del método `__init__`.

3.3. Creando Objetos (Instanciación)

Para crear un objeto a partir de una clase, simplemente "llamamos" a la clase como si fuera una función, pasando los argumentos requeridos por el método `__init__`.

```
```python
```

#### Creamos dos objetos (instancias) de la clase Coche

```
micoche = Coche("Toyota", "Corolla", "Rojo", 50)
cochevecino = Coche("Ford", "Fiesta", "Azul", 30)

print("\n--- Información de los coches ---")
print(f"Mi coche es un {micoche.marca} {micoche.modelo} de color {micoche.color}.")
print(f"El coche de mi vecino es un {cochevecino.marca} {cochevecino.modelo} de color {cochevecino.color}.")
```

#### Accediendo a los atributos de cada objeto

```
print(f"Mi coche tiene {micoche.gasolina} litros de gasolina.")
print(f"El coche de mi vecino tiene {cochevecino.gasolina} litros de gasolina.")
```

**Salida esperada:** ```` ¡Un nuevo Rojo Toyota Corolla ha sido creado! Tiene 50 litros de gasolina y está parado. ¡Un nuevo Azul Ford Fiesta ha sido creado! Tiene 30 litros de gasolina y está parado.`

--- Información de los coches --- Mi coche es un Toyota Corolla de color Rojo. El coche de mi vecino es un Ford Fiesta de color Azul. Mi coche tiene 50 litros de gasolina. El coche de mi vecino tiene 30 litros de gasolina. ```

Como puedes ver, cada objeto (mi\_coche, coche\_vecino) tiene sus propios valores para los atributos definidos en la clase Coche.

---

## 4. Añadiendo Métodos a una Clase

Los métodos son funciones definidas dentro de una clase que operan sobre los atributos del objeto. Al igual que `__init__`, el primer parámetro de cualquier método debe ser `self`.

```
```python class Coche: """Abstracción de los objetos coche."""

    def __init__(self, marca, modelo, color, gasolina_inicial):
        self.marca = marca
        self.modelo = modelo
        self.color = color
        self.gasolina = gasolina_inicial
        self.velocidad_actual = 0
        print(f"¡Un nuevo {self.color} {self.marca} {self.modelo} ha sido creado!")
        print(f"Tiene {self.gasolina} litros de gasolina y está parado.")

    def acelerar(self, incremento_velocidad):
        """Acelera el coche, aumentando su velocidad actual."""
        if self.gasolina > 0:
            self.velocidad_actual += incremento_velocidad
            self.gasolina -= (incremento_velocidad / 10) # Consume gasolina al acelerar
            print(f"El {self.marca} {self.modelo} acelera a {self.velocidad_actual} km/h.")
            print(f"Gasolina restante: {self.gasolina:.2f} litros.")
        else:
            print(f"¡El {self.marca} {self.modelo} no tiene gasolina para acelerar!")

    def frenar(self, decremento_velocidad):
        """Frena el coche, disminuyendo su velocidad actual."""
        self.velocidad_actual = max(0, self.velocidad_actual - decremento_velocidad)
        print(f"El {self.marca} {self.modelo} frena a {self.velocidad_actual} km/h.")

    def repostar(self, cantidad_litros):
        """Añade gasolina al tanque del coche."""
        self.gasolina += cantidad_litros
        print(f"Se han añadido {cantidad_litros} litros. Gasolina actual: {self.gasolina:.2f} litros.")

    def obtener_estado(self):
        """Devuelve una descripción del estado actual del coche."""
        return (f"Estado del {self.marca} {self.modelo}: "
                f"Color: {self.color}, Velocidad: {self.velocidad_actual} km/h, "
                f"Gasolina: {self.gasolina:.2f} litros.")

...`
```

4.1. Invocando Métodos

Para llamar a un método de un objeto, usamos la sintaxis `objeto.nombre_metodo(argumentos)`.

```
```python print("\n--- Operaciones con mi coche ---") micoche = Coche("Tesla", "Model 3", "Blanco", 75) # Un coche eléctrico, pero usaremos
gasolina para el ejemplo micoche.acelerar(50) micoche.acelerar(30) micoche.frenar(20) micoche.repostar(10) # Aunque sea eléctrico,
simulamos la recarga print(micoche.obtener_estado())`
```

```
print("\n--- Operaciones con el coche del vecino ---") cochevecino = Coche("BMW", "X5", "Negro", 20) cochevecino.acelerar(60)
cochevecino.repostar(40) cochevecino.acelerar(40) cochevecino.frenar(100) print(cochevecino.obtener_estado())`
```

**Intentar acelerar sin gasolina**

---

```
print("\n--- Intentando acelerar sin gasolina ---") cochesingasolina = Coche("Fiat", "Panda", "Gris", 0) cochesingasolina.acelerar(10)
cochesingasolina.repostar(5) cochesingasolina.acelerar(10) ````
```

**Salida esperada (parcial, por brevedad):** ```` ... --- Operaciones con mi coche --- ¡Un nuevo Blanco Tesla Model 3 ha sido creado! Tiene 75 litros de gasolina y está parado. El Tesla Model 3 acelera a 50 km/h. Gasolina restante: 70.00 litros. El Tesla Model 3 acelera a 80 km/h. Gasolina restante: 67.00 litros. El Tesla Model 3 frena a 60 km/h. Se han añadido 10 litros. Gasolina actual: 77.00 litros. Estado del Tesla Model 3: Color: Blanco, Velocidad: 60 km/h, Gasolina: 77.00 litros.

... --- Intentando acelerar sin gasolina --- ¡Un nuevo Gris Fiat Panda ha sido creado! Tiene 0 litros de gasolina y está parado. ¡El Fiat Panda no tiene gasolina para acelerar! Se han añadido 5 litros. Gasolina actual: 5.00 litros. El Fiat Panda acelera a 10 km/h. Gasolina restante: 4.00 litros. ````

---

## 5. Conclusión

En esta lección, hemos dado nuestros primeros pasos en el mundo de la Programación Orientada a Objetos. Hemos aprendido que:

- La **POO** es un paradigma que modela el mundo real a través de clases y objetos.
- Una **clase** es una plantilla que define los atributos (estado) y métodos (comportamiento) comunes a un tipo de entidad.
- Un **objeto** es una instancia concreta de una clase, con sus propios valores para los atributos.
- En Python, las clases se definen con `class`.
- El método especial `__init__` actúa como constructor para inicializar los objetos.
- `self` es una referencia a la instancia actual del objeto y es fundamental para acceder a sus atributos y métodos.
- Los métodos son funciones dentro de una clase que permiten a los objetos realizar acciones.

Comprender estos principios es fundamental para construir aplicaciones más robustas, modulares y fáciles de mantener. En las próximas lecciones, exploraremos otros pilares de la POO como la encapsulación, la herencia y el polimorfismo.

---

## ¡Tu Turno! (Ejercicio Práctico)

Para solidificar tu aprendizaje, te propongo el siguiente ejercicio:

### Crea una clase Libro:

- Debe tener un constructor `__init__` que reciba `título`, `autor` y `num_paginas`.
- Debe tener un atributo `leído` inicializado a `False`.
- Añade un método `marcar_como_leído()` que cambie el atributo `leído` a `True` y muestre un mensaje.
- Añade un método `obtener_informacion()` que devuelva una cadena con el título, autor, número de páginas y si ha sido leído o no.

### 2. Crea al menos dos objetos Libro diferentes.

#### Realiza las siguientes operaciones:

- Imprime la información de ambos libros.
- Marca uno de los libros como leído.
- Imprime nuevamente la información de ambos libros para ver el cambio.

¡Mucha suerte y no dudes en experimentar!

## 3.2 Lección 4.2: Herencia y Polimorfismo (Conceptos Básicos)

¡Absolutamente! Como tu profesor experto, te guiaré a través de los conceptos fundamentales de la Herencia y el Polimorfismo en la Programación Orientada a Objetos (POO). Prepárate para descubrir cómo estos pilares te permitirán escribir código más reutilizable, flexible y fácil de mantener.

---

## Lección 4.2: Herencia y Polimorfismo (Conceptos Básicos)

---

### Introducción: Los Pilares de la Reutilización y la Flexibilidad

¡Bienvenidos a la Lección 4.2! En el mundo de la Programación Orientada a Objetos (POO), la **Herencia** y el **Polimorfismo** son dos conceptos poderosos que nos permiten construir sistemas complejos de manera eficiente. La herencia nos ayuda a reutilizar código y establecer relaciones lógicas entre nuestras clases, mientras que el polimorfismo nos brinda la flexibilidad de tratar objetos de diferentes tipos de una

manera uniforme.

En esta lección, exploraremos cómo aplicar la herencia para evitar la repetición de código y cómo el polimorfismo nos permite diseñar sistemas más dinámicos y extensibles.

---

## 1. Herencia: Reutilizando el ADN del Código

---

Imagina que estás diseñando un sistema para gestionar diferentes tipos de animales. Todos los animales tienen un nombre y pueden "saludar", pero un perro ladra y un gato maúlla. En lugar de definir el nombre y el método `saludar` en cada clase de animal, la herencia nos permite definir estas características comunes una sola vez y luego hacer que las clases específicas las "hereden".

### 1.1 ¿Qué es la Herencia?

La **herencia** es un mecanismo que permite a una clase (llamada **clase hija**, **subclase** o **clase derivada**) adquirir las propiedades (atributos) y comportamientos (métodos) de otra clase (llamada **clase padre**, **superclase** o **clase base**).

La relación que establece la herencia es una relación "es un" (is-a). Por ejemplo, un Perro *es un* Animal, un Coche *es un* Vehículo.

**Beneficios clave:**

- \* **Reutilización de código:** Evita la duplicación de código al compartir funcionalidades comunes.
- \* **Extensibilidad:** Permite añadir nuevas funcionalidades o especializar las existentes sin modificar el código original de la clase padre.
- \* **Organización:** Ayuda a estructurar el código de manera lógica y jerárquica.

### 1.2 Sintaxis Básica de la Herencia

En Python, la sintaxis para la herencia es muy sencilla:

```
```python
class ClasePadre:
    # Atributos y métodos comunes
    def __init__(self, nombre):
        self.nombre = nombre

    def saludar(self):
        return f"Hola, soy {self.nombre}."

class ClaseHija(ClasePadre):
    # La clase hija hereda de ClasePadre
    # Atributos y métodos específicos de la hija
    def __init__(self, nombre, raza):
        # Llamamos al constructor de la clase padre para inicializar sus atributos
        super().__init__(nombre)
        self.raza = raza

    def ladrar(self):
        return "¡Guau! ¡Guau!"
```

Ejemplo de uso

```
miperro = ClaseHija("Fido", "Labrador")
print(miperro.saludar()) # Método heredado de ClasePadre
print(miperro.ladrar()) # Método propio de ClaseHija
```

Explicación:

- * `class ClaseHija(ClasePadre):` indica que ClaseHija hereda de ClasePadre.
- * `super().__init__(nombre):` La función `super()` se utiliza para llamar a métodos de la clase padre. En este caso, llamamos al constructor `__init__` de ClasePadre para asegurarnos de que el atributo `nombre` se inicialice correctamente. Es una práctica fundamental al trabajar con herencia.

1.3 Sobreescritura de Métodos (Method Overriding)

Una clase hija puede proporcionar su propia implementación de un método que ya existe en su clase padre. Esto se conoce como **sobreescritura de métodos**. La clase hija "sobreescrive" el comportamiento del padre para ese método específico.

```
```python
class Animal:
 def __init__(self, nombre):
 self.nombre = nombre

 def hacer_sonido(self):
 return "Sonido genérico de animal"

class Perro(Animal):
 def hacer_sonido(self):
 # Sobreescrive el método hacer_sonido de Animal
 return "¡Guau! ¡Guau!"

class Gato(Animal):
 def hacer_sonido(self):
 # Sobreescrive el método hacer_sonido de Animal
 return "¡Miau! ¡Miau!"
```

### Ejemplo de uso

---

```
animalgenerico = Animal("Desconocido") perroobj = Perro("Max") gato_obj = Gato("Whiskers")
```

```
print(f'{animalgenerico.nombre}: {animalgenerico.hacersonido()}') print(f'{perroobj.nombre}: {perroobj.hacersonido()}') print(f'{gatoobj.nombre}: {gatoobj.hacersonido()}') `` En este ejemplo, aunque Perro y Gato heredan de Animal, cada uno implementa su propio hacersonido`, demostrando la sobrescritura.
```

## 1.4 Herencia Múltiple (¡Cuidado!)

Python permite que una clase herede de múltiples clases padre. Esto se conoce como **herencia múltiple**. La sintaxis es simplemente listar las clases padre separadas por comas:

```
python class Cocodrilo(Terrestre, Acuatico): pass
```

**El Orden de Resolución de Métodos (MRO - Method Resolution Order):** Cuando una clase hija hereda de múltiples padres y estos padres tienen métodos con el mismo nombre, Python necesita una regla para decidir qué implementación usar. Esta regla se conoce como MRO. En Python, el MRO sigue un algoritmo que, simplificando, da **prioridad a las clases que se encuentran más a la izquierda** en la definición de la herencia.

Veamos el ejemplo que nos proporcionaste:

```
``python class Terrestre: def desplazar(self): print("El animal anda")
```

```
class Acuatico: def desplazar(self): print("El animal nada")
```

```
class Cocodrilo(Terrestre, Acuatico): pass
```

```
c = Cocodrilo() c.desplazar() ``
```

**Análisis:** 1. La clase Cocodrilo hereda de Terrestre y Acuatico. 2. Ambas clases padre (Terrestre y Acuatico) tienen un método llamado desplazar. 3. Cuando se llama a c.desplazar(), Python busca el método desplazar en el MRO de Cocodrilo. 4. Como Terrestre está definida **antes** que Acuatico en la lista de herencia de Cocodrilo, la implementación de desplazar de Terrestre es la que prevalece.

**Salida esperada:** El animal anda

**Importante:** La herencia múltiple puede hacer que el código sea más complejo y difícil de entender o depurar debido a la complejidad del MRO. A menudo, se prefieren otras técnicas como la composición o el uso de *mixins* para lograr una funcionalidad similar de una manera más clara.

---

## 2. Polimorfismo: Muchas Formas para un Mismo Mensaje

La palabra **polimorfismo** proviene del griego "poly morphos", que significa "varias formas". En POO, el polimorfismo se refiere a la habilidad de objetos de distintas clases de responder al mismo mensaje (es decir, a la llamada de un método) de maneras diferentes, siempre y cuando compartan una interfaz común (es decir, el mismo nombre de método).

### 2.1 ¿Qué es el Polimorfismo?

Imagina que tienes una lista de animales diferentes: un perro, un gato y un pato. Si quieres que todos hagan su sonido característico, no necesitas saber si es un perro o un gato; simplemente les dices "haz tu sonido". Cada animal responderá de su propia manera.

El polimorfismo nos permite escribir código más genérico y flexible. Podemos interactuar con objetos de diferentes clases a través de una interfaz común, sin necesidad de conocer su tipo exacto en tiempo de ejecución.

### 2.2 Polimorfismo en Acción (Duck Typing)

Python implementa el polimorfismo principalmente a través de lo que se conoce como **Duck Typing** (tipado de pato). La filosofía es: "Si camina como un pato y grazna como un pato, entonces es un pato". Esto significa que no nos importa el tipo explícito de un objeto, sino si tiene los métodos que necesitamos llamar.

Volvamos a nuestro ejemplo de animales con el método hacer\_sonido:

```
``python class Animal: def init(self, nombre): self.nombre = nombre
```

```
def hacer_sonido(self):
 return "Sonido genérico de animal"
```

```
class Perro(Animal): def hacer_sonido(self): return "¡Guau! ¡Guau!"
```

```
class Gato(Animal): def hacer_sonido(self): return "¡Miau! ¡Miau!"
```

## Una función que acepta cualquier objeto que tenga el método 'hacer\_sonido'

---

```
def escuchar_sonido(animal): print(f"{animal.nombre} dice: {animal.hacer_sonido()}")
```

## Creamos instancias de diferentes clases

---

```
animal_generico = Animal("Desconocido") perro_obj = Perro("Max") gato_obj = Gato("Whiskers")
```

## Llamamos a la misma función con diferentes tipos de objetos

---

```
escuchar_sonido(animal_generico) escuchar_sonido(perro_obj) escuchar_sonido(gato_obj)
```

## Incluso una clase que no hereda de Animal puede ser "polimórfica"

---

### si implementa el método requerido.

---

```
class Campana: def __init__(self, nombre="Campana"): self.nombre = nombre def hacer_sonido(self): return "¡Ding! ¡Dong!"
```

```
micampana = Campana() escuchar_sonido(micampana) # Funciona porque Campana tiene 'hacer_sonido' ```
```

**Explicación:** La función `escuchar_sonido` no sabe ni le importa si el objeto que recibe es un `Animal`, un `Perro`, un `Gato` o una `Campana`. Solo espera que el objeto tenga un método `hacer_sonido()`. Esta es la esencia del polimorfismo: la capacidad de tratar objetos de diferentes clases de manera uniforme a través de una interfaz común.

---

## 3. ¿Por qué son importantes la Herencia y el Polimorfismo?

---

- **Reutilización de Código:** La herencia reduce la duplicación, lo que significa menos código que escribir y mantener.
  - **Mantenibilidad:** Los cambios en la clase base se propagan automáticamente a las clases derivadas (a menos que se sobrescriban), lo que simplifica las actualizaciones.
  - **Extensibilidad:** Es fácil añadir nuevas funcionalidades o tipos de objetos sin modificar el código existente.
  - **Flexibilidad:** El polimorfismo permite que tu código sea más adaptable a diferentes tipos de datos, haciendo que las funciones y métodos sean más genéricos.
  - **Código más Limpio y Legible:** Al organizar las clases en jerarquías lógicas, el código se vuelve más estructurado y fácil de entender.
- 

## 4. Conclusión

---

Hemos explorado la **Herencia** como un mecanismo para la reutilización de código y la creación de jerarquías de clases "es un". Aprendimos sobre la sobrescritura de métodos y la herencia múltiple, con su importante consideración del MRO. Luego, profundizamos en el **Polimorfismo**, la habilidad de objetos de diferentes clases para responder al mismo mensaje de maneras únicas, facilitando un código más flexible y genérico, especialmente a través del *Duck Typing* en Python.

Dominar estos conceptos te permitirá diseñar sistemas robustos, escalables y fáciles de mantener, que son la base de la programación orientada a objetos moderna.

---

## 5. Ejercicios Propuestos

---

Para consolidar tu aprendizaje, te propongo los siguientes ejercicios:

**Jerarquía de Vehículos:**

- Crea una clase base `Vehiculo` con atributos como `marca`, `modelo` y un método `moverse()` que imprima "El vehículo se está moviendo".
- Crea dos clases hijas: `Coche` y `Bicicleta`.
- `Coche` debe tener un atributo adicional `numero_puertas` y sobrescribir `moverse()` para imprimir "El coche [marca] [modelo] está circulando".
- `Bicicleta` debe tener un atributo adicional `tipo_bicicleta` (ej. "montaña", "carretera") y sobrescribir `moverse()` para imprimir "La bicicleta [marca] [modelo] está pedaleando".
- Crea instancias de `Vehiculo`, `Coche` y `Bicicleta` y llama a su método `moverse()`.

#### Polimorfismo con Vehículos:

- Crea una función llamada `simular_movimiento(vehiculo)` que tome un objeto `vehiculo` como parámetro y llame a su método `moverse()`.
- Pasa a esta función las instancias de `Vehiculo`, `Coche` y `Bicicleta` que creaste en el ejercicio anterior. Observa cómo la misma función puede interactuar con diferentes tipos de objetos de manera polimórfica.

¡Mucha suerte con tus ejercicios y sigue practicando!

### 3.3 Lección 4.3: Manejo de Excepciones

¡Excelente! Bienvenidos a la Lección 4.3 de nuestro curso. Hoy abordaremos un pilar fundamental en la construcción de software robusto y fiable: el **Manejo de Excepciones**. Como desarrolladores, sabemos que los errores son una parte inevitable del proceso. La clave no es evitarlos por completo, sino saber cómo gestionarlos cuando ocurren, para que nuestro programa no colapse y pueda recuperarse elegantemente.

---

## Lección 4.3: Manejo de Excepciones

---

### 1. Introducción: La Realidad de los Errores

En el mundo de la programación, un "error" o "excepción" es un evento que interrumpe el flujo normal de ejecución de un programa. Esto puede deberse a diversas razones: un archivo que no existe, una división por cero, un tipo de dato inesperado, o incluso un problema de red. Sin un manejo adecuado, estas situaciones pueden llevar a que nuestro programa se detenga abruptamente, ofreciendo una experiencia de usuario deficiente y, en entornos críticos, consecuencias graves.

El manejo de excepciones nos permite "atrapar" estos errores, reaccionar ante ellos de manera controlada y, si es posible, continuar la ejecución del programa o al menos terminarlo de forma limpia. Esto mejora significativamente la **fiabilidad** y la **robustez** de nuestro software.

### 2. El Bloque try-except: La Primera Línea de Defensa

El mecanismo fundamental para el manejo de excepciones en muchos lenguajes, incluido Python, es el bloque try-except.

#### 2.1. Estructura Básica

La idea es simple: colocamos el código que *podría* generar una excepción dentro de un bloque try. Si una excepción ocurre dentro de este bloque, la ejecución del try se detiene y el control pasa a un bloque except asociado, que contiene el código para manejar ese error.

```
python try: # Código que podría generar una excepción resultado = 10 / 0 # Esto causará un ZeroDivisionError
except: # Código que se ejecuta si ocurre CUALQUIER excepción en el bloque try
print("¡Ocurrió un error inesperado!")
```

#### 2.2. Flujo de Ejecución

Cuando se lanza una excepción en el bloque try, el intérprete busca en cada una de las cláusulas except un manejador adecuado para el tipo de error que se produjo.

- Si se encuentra un except que coincide con el tipo de excepción (o un except genérico sin tipo especificado), el código dentro de ese except se ejecuta.
- Si no se encuentra ningún manejador adecuado, la excepción se **propaga** hacia arriba en la pila de llamadas. Si no se maneja en ningún nivel, el programa terminará con un error no controlado.

### 3. Manejo de Excepciones Específicas

---

Es una buena práctica ser lo más específico posible al manejar excepciones. Esto nos permite reaccionar de manera diferente a distintos tipos de errores y evita "silenciar" errores que no esperábamos.

```
python try: numero = int("abc") # Esto causará un ValueError except ValueError: print("Error: No se pudo convertir el texto a un número entero.") except ZeroDivisionError: print("Error: ¡No puedes dividir por cero!") except Exception as e: # Captura cualquier otra excepción no especificada print(f'Ocurrió un error inesperado: {e}')
```

En este ejemplo, si `int("abc")` falla, solo se ejecutará el primer `except ValueError`. Si intentáramos dividir por cero, se ejecutaría el segundo `except ZeroDivisionError`. La cláusula `except Exception as e:` es un "comodín" que captura cualquier otra excepción no manejada específicamente, y nos permite acceder al objeto de la excepción (`e`) para obtener más detalles.

### 4. Manejo de Múltiples Excepciones con una Tupla

---

Podemos hacer que un mismo `except` sirva para tratar más de una excepción usando una tupla para listar los tipos de error que queremos que trate el bloque. Esto es útil cuando la lógica de manejo es la misma para varios tipos de errores.

```
python try: num = int("3a") # Esto lanzará un ValueError # print(no_existe) # Si la línea anterior no falla, esta lanzaría un NameError except (NameError, ValueError): print("Ocurrió un error de nombre o de valor. Por favor, revisa tu entrada.")
```

```
print("El programa continúa después del bloque try-except.")
```

En el ejemplo anterior, `int("3a")` generará un `ValueError`. El bloque `except (NameError, ValueError)` lo atrapará y ejecutará su código. La línea `print(no_existe)` nunca se alcanzará porque la excepción `ValueError` detiene la ejecución del `try` inmediatamente.

### 5. La Cláusula `else`: Cuando Todo Va Bien

---

La construcción `try-except` puede contar además con una cláusula `else`. Esta define un fragmento de código a ejecutar **solo si no se ha producido ninguna excepción** en el bloque `try`. Es el lugar ideal para el código que debe ejecutarse si la operación "peligrosa" fue exitosa.

```
python try: x = 10 y = 2 resultado = x / y except ZeroDivisionError: print("¡Error: División por cero!") else: print(f'La división se realizó con éxito. Resultado: {resultado}') print("Todo está bien, no hubo errores en el bloque try.")
```

En este caso, como `10 / 2` no genera un `ZeroDivisionError`, el bloque `else` se ejecutará. Si `y` fuera `0`, el `except` se activaría y el `else` sería ignorado.

### 6. La Cláusula `finally`: Siempre se Ejecuta

---

También existe una cláusula `finally` que se ejecuta **siempre**, se produzca o no una excepción, y sin importar si un `except` o `else` se ejecutó. Esta cláusula se suele utilizar, entre otras cosas, para tareas de limpieza.

Imagina que abres un archivo o una conexión de red. Es crucial que estos recursos se cierren, incluso si ocurre un error durante su uso. `finally` garantiza esta limpieza.

```
python try: archivo = open("mi_archivo.txt", "r") contenido = archivo.read() print(contenido) # raise ValueError("Simulando un error después de leer") # Descomentar para probar con error except FileNotFoundError: print("Error: El archivo no fue encontrado.") except Exception as e: print(f'Ocurrió un error inesperado: {e}') finally: # Este bloque siempre se ejecuta if 'archivo' in locals() and not archivo.closed: archivo.close() print("El archivo ha sido cerrado.") else: print("No se abrió ningún archivo o ya estaba cerrado.")
```

```
print("El programa ha terminado su ejecución.")
```

En este ejemplo, el `finally` asegura que `archivo.close()` se llame, ya sea que la lectura del archivo haya sido exitosa, que el archivo no se haya encontrado (`FileNotFoundError`), o que cualquier otra excepción haya ocurrido.

### 7. Resumen del Flujo de Ejecución

---

Para consolidar, el orden de ejecución en un bloque `try-except-else-finally` es el siguiente:

**try:** Se ejecuta el código dentro de este bloque.

**Si no hay excepción:**

- Se ejecuta el bloque `else`.
- Finalmente, se ejecuta el bloque `finally`.

**Si hay una excepción:**

- La ejecución del `try` se detiene.

Se busca un bloque `except` que coincida con el tipo de excepción.

- Si se encuentra, se ejecuta el bloque `except` correspondiente.
- Si no se encuentra, la excepción se propaga.
- Finalmente, se ejecuta el bloque `finally` (incluso si la excepción se propagó).

## 8. Buenas Prácticas en el Manejo de Excepciones

---

- **Sé específico:** Atrapa solo las excepciones que esperas y sabes cómo manejar. Evita `except :` sin especificar el tipo de excepción, ya que puede ocultar errores importantes.
- **Maneja, no solo silencies:** Cuando atrapas una excepción, haz algo útil: registra el error, informa al usuario, intenta recuperarte, o al menos termina limpiamente. No uses `pass` en un `except` a menos que tengas una razón muy específica y documentada.
- **Usa `else` para el éxito:** Coloca el código que depende del éxito del `try` en la cláusula `else`.
- **Usa `finally` para la limpieza:** Garantiza que los recursos (archivos, conexiones) se liberen, sin importar el resultado del `try`.
- **Registra los errores:** En aplicaciones reales, es crucial registrar los detalles de las excepciones (tipo, mensaje, pila de llamadas) para depuración y monitoreo.

---

Con esto, hemos cubierto los fundamentos del manejo de excepciones en Python. Dominar estas herramientas es esencial para escribir código robusto que pueda resistir los desafíos del mundo real. ¡Ahora, a practicar!